

AI 프로젝트 진행 가이드

3단계 · 15일 로드맵 정리

이 문서는 강사님이 제공한 강의자료를 프로젝트 폴더 구성 순서(1단계 기획 → 2단계 데이터/DB → 3단계 서비스설계·프로토타입, 각 단계 Day01~Day05)에 맞춰 원문 내용을 그대로 재배열한 자료입니다. 내용을 요약·축약하지 않고, 각 Day에 해당하는 강의자료 전체를 정리했습니다. 실제 진행 기간은 3단계 × 5일 = 총 15일(약 3주)입니다.

전체 로드맵 한눈에 보기

단계	폴더명	핵심 산출물
1단계	day01_시프로젝트개요	프로젝트 방향 이해
1단계	day02_시장조사_문제정의	시장조사 보고서, Pain Point 정의
1단계	day03_요구사항분석	데이터 요구사항 목록
1단계	day04_가설수립_KPI설계	가설 문장, KPI 지표
1단계	day05_요구사항정의서	요구사항정의서
2단계	day01_데이터개요	데이터 이해 체크리스트
2단계	day02_데이터전처리	데이터 품질 기준
2단계	day03_데이터정의서	데이터 정의서
2단계	day04_데이터모델링	ERD, 정규화된 테이블 설계
2단계	day05_API정의서	API 정의서
3단계	day01_MVP정의	User Story, MoSCoW 우선순위
3단계	day02_정보구조(IA)	IA 구조도, User Flow
3단계	day03_화면설계서	화면설계서(와이어프레임+기능명세)
3단계	day04_서비스시나리오	정상/예외 흐름, 오류 메시지 설계
3단계	day05_웹_프로토타입구현	프로토타입 + 검증 결과

1단계. AI 프로젝트 기획 (day01 ~ day05)

Day 01 — AI프로젝트개요

이 폴더(day01_AI프로젝트개요)에 해당하는 별도 강의자료는 이번에 전달받지 못했습니다. 강사님께 자료 존재 여부를 확인해 주세요. 다만 이후 자료들(요구사항정의서.md)에 아래와 같은 참고 흐름이 제시되어 있어 프로젝트 전체 그림을 잡는 데 참고할 수 있습니다.

[참고영상] 바이트코딩 직전까지의 기획 계획 과정

1. 문제 정의
2. AI로 시장조사 하는 법
3. 패턴 찾기
4. 핵심 가치 찾기
5. PRD 작성
6. 실패 가능성 묻기
7. 암묵적 가정 묻기
8. 마케팅 타겟 정하기
9. 예외 케이스

Day 02 — 시장조사 · 문제정의

자료 출처: 1_시장조사.md, 2_문제_정의_Pain_Point_.md

시장조사와 경쟁사 분석

시장조사는 “무엇을 만들지”를 결정하기 전에, 사용자가 실제로 겪는 문제와 시장의 빈틈을 확인하는 과정입니다.

오늘의 학습 목표

- 시장조사가 왜 필요한지 설명할 수 있다.
- 시장조사에서 확인해야 할 항목을 구분할 수 있다.
- 경쟁사 분석을 통해 차별화 기회를 찾을 수 있다.
- 조사 결과를 문제 정의와 서비스 기획으로 연결할 수 있다.

시장조사란?

시장조사는 단순히 경쟁사를 찾아보는 일이 아닙니다.

다음 질문에 답하기 위한 기획 활동입니다. - 사용자는 어떤 문제를 겪고 있는가? - 시장은 충분히 크거나 성장 가능성이 있는가? - 기존 서비스는 어떤 방식으로 문제를 해결하고 있는가? - 아직 해결되지 않은 불편함은 무엇인가? - 우리 서비스가 차별화할 수 있는 지점은 어디인가?

왜 시장조사가 필요한가?

아이디어만으로 서비스를 만들면 다음 문제가 생깁니다.

- 실제 사용자가 원하지 않는 기능을 만들 수 있다.
- 이미 비슷한 서비스가 많은데 차별점이 없을 수 있다.
- 시장 규모나 사용자 니즈를 과대평가할 수 있다.
- 해결책부터 정해 놓고 문제를 끼워 맞출 수 있다.

시장조사는 “이 아이디어가 정말 필요한가?”를 확인하는 과정입니다.

시장조사에서 확인할 4가지

구분	확인할 내용	핵심 질문
시장	규모, 성장성, 변화 흐름	이 영역은 커지고 있는가?
사용자	대상, 행동, 불편함	누가 어떤 상황에서 불편한가?
경쟁사	제공 기능, 강점, 약점	기존 서비스는 무엇을 잘하고 못하는가?
기회	미해결 문제, 차별화 지점	우리가 새롭게 제공할 가치는 무엇인가?

시장 흐름을 볼 때의 질문

- 최근 사용자의 관심이 증가하고 있는가?
- 관련 서비스나 상품이 늘어나고 있는가?
- 온라인 이용, 모바일 이용, AI 활용 등 행동 변화가 있는가?
- 정책, 규제, 데이터 공개 등 외부 환경 변화가 있는가?
- 앞으로도 지속될 변화인가, 일시적인 유행인가?

사용자 조사의 핵심

사용자 조사는 “사람들이 무엇을 원하는가”보다,

“어떤 상황에서 무엇이 불편한가”

를 찾는 것이 중요합니다.

확인할 내용 - 사용자는 누구인가? - 어떤 상황에서 서비스를 필요로 하는가? - 현재는 문제를 어떻게 해결하고 있는가? - 그 과정에서 시간이 오래 걸리거나 어렵거나 불안한 지점은 무엇인가?

경쟁사 분석의 목적

경쟁사 분석은 누가 더 좋은지 순위를 매기는 일이 아닙니다.

목적은 다음과 같습니다. - 기존 서비스가 이미 해결한 문제를 확인한다. - 기존 서비스가 아직 해결하지 못한 문제를 찾는다. - 사용자가 당연하게 기대하는 기본 기능을 파악한다. - 새 서비스가 차별화할 수 있는 영역을 찾는다.

경쟁사 분석 항목

항목	확인할 질문
대상 사용자	누구를 주요 고객으로 보는가?
핵심 기능	어떤 기능을 제공하는가?
사용 경험	찾기 쉽고 이해하기 쉬운가?
정보 제공	사용자가 판단할 만큼 충분한 정보를 주는가?
신뢰 요소	리뷰, 인증, 출처, 설명이 있는가?
약점	사용자가 여전히 불편해할 부분은 무엇인가?

경쟁사 비교표 작성법

경쟁사 비교표는 기능을 많이 적는 것보다,

사용자의 문제 해결에 중요한 기준을 고르는 것

이 더 중요합니다.

비교 기준	서비스 A	서비스 B	서비스 C	기획 중인 서비스의 기회
검색 편의성	O	O	△	조건별 탐색 강화
정보 이해도	△	O	△	쉬운 설명 제공
개인화	X	△	X	맞춤 추천 제공
사용 후 관리	X	X	△	지속 관리 기능 제공

차별화 기회 찾기

경쟁사를 조사한 뒤에는 다음 문장으로 정리합니다.

기존 서비스는 _____을 잘 제공하지만,
사용자는 여전히 _____에서 불편함을 겪는다.
따라서 새 서비스는 _____을 제공할 기회가 있다.

조사 결과 해석하기

자료를 많이 모으는 것보다 해석이 중요합니다.

사실
↓
의미

예시 - 사실: 사용자는 정보를 여러 곳에서 찾아본다. - 의미: 한 번에 판단하기 어렵다. - 기획 판단: 비교, 요약, 추천 기능이 필요할 수 있다.

시장조사 결과물의 기본 구조

시장조사서는 다음 흐름으로 정리하면 읽기 쉽습니다.

1. 시장 현황
2. 사용자 행동과 문제
3. 경쟁사 분석
4. 경쟁사 비교표
5. 미해결 문제
6. 기획 기회

흔한 실수

- 검색 결과를 그대로 붙여 넣고 해석하지 않는다.
- 경쟁사의 장점만 나열하고 약점을 찾지 않는다.
- 사용자의 문제보다 만들고 싶은 기능을 먼저 적는다.
- 시장 규모가 크다는 이유만으로 서비스가 필요하다고 판단한다.
- 조사 결과가 다음 단계의 문제 정의로 연결되지 않는다.

오늘 기억할 문장

좋은 시장조사는 자료 수집이 아니라, 사용자 문제와 서비스 기회를 찾는 과정입니다.

다음 단계에서는 시장조사 결과를 바탕으로 Pain Point를 정의합니다.

문제 정의와 Pain Point

AI 서비스 기획은 “어떤 AI를 만들까?”가 아니라 “어떤 문제를 해결할까?”에서 시작합니다.

오늘의 학습 목표

- 문제 정의가 왜 중요한지 설명할 수 있다.
- Pain Point와 Solution의 차이를 구분할 수 있다.
- 좋은 Pain Point의 조건을 이해할 수 있다.
- 문제 정의 문장을 작성하는 방법을 익힐 수 있다.

AI 프로젝트의 시작

많은 사람이 이렇게 생각합니다.

AI 기술
↓
서비스

하지만 실제 기획 흐름은 다릅니다.

실제 AI 서비스 기획 흐름

문제 발견
↓
사용자 이해
↓

시장 조사
↓
Pain Point 정의
↓
AI 활용 가능성 검토
↓
서비스 기획

AI는 출발점이 아니라, 문제를 해결하기 위한 수단입니다.

왜 문제 정의가 중요한가?

좋은 AI도 잘못된 문제를 해결하면 성공하기 어렵습니다.

예를 들어,

“챗봇을 만들자.”

이 문장은 문제 정의가 아닙니다.

왜냐하면 사용자가 무엇 때문에 불편한지 설명하지 않기 때문입니다.

문제와 해결책의 차이

구분	의미	예시
문제	사용자가 겪는 불편함	고객 문의 답변을 받기까지 시간이 오래 걸린다.
해결책	문제를 해결하기 위한 방법	AI 챗봇을 만든다.
기능	해결책을 서비스 안에서 구현한 형태	자주 묻는 질문 자동 응답

잘못된 문제 정의

다음 문장은 대부분 해결책에 가깝습니다.

- AI 챗봇 만들기
- 추천 AI 만들기
- 자동 요약 기능 만들기
- 개인화 서비스 만들기

“무엇을 만들지”보다 “왜 필요한지”가 먼저 나와야 합니다.

올바른 문제 정의

좋은 문제 정의는 사용자의 불편함을 설명합니다.

- 사용자가 필요한 정보를 찾는 데 시간이 오래 걸린다.
- 선택지가 너무 많아 무엇을 골라야 할지 어렵다.
- 정보가 여러 곳에 흩어져 있어 비교하기 어렵다.
- 전문 용어가 많아 내용을 이해하기 어렵다.
- 사용 후에도 계속 관리하기 어렵다.

Pain Point란?

Pain Point는 사용자가 불편함을 느끼거나 해결하고 싶은 문제입니다.

사용자가

"이거 정말 불편한데..."

AI 기획자는 Pain Point를 발견하고, AI가 그 문제 해결에 도움이 되는지 판단합니다.

Pain Point가 중요한 이유

- 사용자의 실제 니즈를 확인할 수 있다.
- 서비스 기능의 우선순위를 정할 수 있다.
- AI가 필요한 문제와 필요하지 않은 문제를 구분할 수 있다.
- 기획자, 디자이너, 개발자가 같은 방향으로 일할 수 있다.

Pain Point의 유형

유형	의미	확인 질문
시간(Time)	시간이 오래 걸리는 문제	사용자가 오래 기다리거나 반복하는 일이 있는가?
비용(Cost)	돈, 노력, 자원이 많이 드는 문제	사용자가 불필요한 비용을 쓰고 있는가?
정보(Information)	정보가 부족하거나 너무 많은 문제	사용자가 판단에 필요한 정보를 이해할 수 있는가?
경험(Experience)	이용 과정이 불편한 문제	사용자가 중간에 포기하거나 헛갈리는 지점이 있는가?
신뢰(Risk)	불안하거나 믿기 어려운 문제	사용자가 결과를 신뢰할 근거가 있는가?

좋은 Pain Point의 조건

좋은 Pain Point는 다음 조건을 만족합니다.

- 실제 사용자가 겪는 문제이다.
- 구체적인 상황이 드러난다.
- 해결했을 때 사용자에게 가치가 있다.
- 해결 여부를 어느 정도 확인할 수 있다.
- AI 또는 데이터 활용 가능성을 검토할 수 있다.

문제 정의 문장 공식

문제 정의는 다음 형태로 작성하면 명확해집니다.

[사용자]는 [상황]에서
[문제] 때문에
[불편함 또는 손실]을 겪는다.

문제 정의 예시

나쁜 예시

“AI 추천 기능이 필요하다.”

좋은 예시

“초보 사용자는 선택지가 많은 상황에서 자신에게 맞는 대상을 고르기 어려워 결정에 시간이 오래 걸린다.”

Solution과 Pain Point 구분하기

Pain Point	Solution
정보를 찾는 데 시간이 오래 걸린다.	AI 검색/요약
선택지가 많아 결정하기 어렵다.	AI 맞춤 추천

설명이 어려워 이해하기 힘들다.	쉬운 용어 변환
리뷰가 많아 핵심을 파악하기 어렵다.	AI 리뷰 요약
사용 후 관리가 어렵다.	알림 및 관리 기능

문제를 먼저 정의하고, 그다음 해결책을 고민해야 합니다.

어떤 문제에 AI가 적합한가?

AI는 다음 유형의 문제에 잘 맞습니다.

- 많은 정보를 요약해야 하는 문제
- 사용자의 조건에 따라 추천이 달라지는 문제
- 자연어로 질문하고 답변해야 하는 문제
- 여러 데이터를 비교해 판단을 도와야 하는 문제
- 반복적인 상담이나 안내가 필요한 문제

AI가 꼭 필요하지 않은 경우

다음 문제는 일반적인 기능 설계로 충분할 수 있습니다.

- 단순한 버튼 클릭
- 정해진 규칙에 따른 계산
- 고정된 목록 조회
- 회원가입, 로그인, 결제 같은 기본 기능

AI를 쓰는 것이 목적이 아니라, 문제 해결에 도움이 되는지가 중요합니다.

문제 우선순위 정하기

모든 문제를 한 번에 해결할 수는 없습니다.

우선순위는 다음 기준으로 판단합니다.

기준	질문
심각도	이 문제가 사용자에게 얼마나 큰 불편을 주는가?
빈도	얼마나 자주 발생하는가?
대상 규모	많은 사용자가 겪는 문제인가?
해결 가능성	현재 기술과 데이터로 해결 가능한가?
서비스 가치	해결하면 서비스 차별화에 도움이 되는가?

흔한 실수

- 문제보다 기능을 먼저 정한다.
- 사용자를 너무 넓게 잡는다.
- “불편하다”라고만 쓰고 상황을 설명하지 않는다.
- AI가 필요한 이유를 검토하지 않는다.
- 해결할 문제를 너무 많이 잡아 초점이 흐려진다.

오늘 기억할 문장

Pain Point는 기능 목록이 아니라, 사용자가 실제로 겪는 불편함입니다.

시장조사에서 발견한 사실을 Pain Point로 정리하면, 다음 단계의 요구사항 정의가 훨씬 쉬워집니다.

Day 03 — 요구사항분석

자료 출처: 데이터_요구사항_분석.md

데이터 요구사항 분석

AI 서비스 기획자는 기능을 정의한 뒤, 그 기능이 실제로 동작하기 위해 **어떤 데이터가 필요한지** 설명할 수 있어야 합니다.

오늘의 학습 목표

- 데이터 요구사항 분석의 목적을 설명할 수 있다.
- 기능별로 필요한 데이터를 구분할 수 있다.
- 공공데이터와 API의 기본 개념을 이해할 수 있다.
- 데이터 활용 가능성을 검토하는 기준을 설명할 수 있다.

데이터 요구사항 분석이란?

데이터 요구사항 분석은 서비스가 동작하기 위해 필요한 데이터를 정의하는 과정입니다.

사용자 문제
↓
서비스 기능
↓
필요 데이터
↓
데이터 출처
↓
활용 가능성 검토

왜 필요한가?

AI 서비스는 아이디어만으로 동작하지 않습니다.

AI가 추천, 요약, 검색, 상담, 분류, 예측을 하려면 근거가 되는 데이터가 필요합니다.

- 추천하려면 추천 기준 데이터가 필요하다.
- 요약하려면 요약할 원문 데이터가 필요하다.
- 검색하려면 검색 대상 데이터가 필요하다.
- 상담하려면 답변 근거 데이터가 필요하다.
- 개인화하려면 사용자 조건 데이터가 필요하다.

기획자의 역할

AI 기획자가 모든 데이터를 직접 수집하거나 처리할 필요는 없습니다.

하지만 다음은 정의할 수 있어야 합니다.

- 어떤 데이터가 필요한가?
- 그 데이터는 왜 필요한가?
- 어디에서 확보할 수 있는가?
- 어떤 형태로 제공되는가?
- 서비스 기능과 어떻게 연결되는가?
- 사용해도 되는 데이터인가?

데이터 요구사항의 기본 질문

데이터를 정리하기 전, 다음 질문을 던집니다.

질문	의미
어떤 문제를 해결하려는가?	데이터 요구의 출발점
어떤 기능이 필요한가?	데이터가 쓰일 위치
기능이 판단해야 할 것은 무엇인가?	필요한 데이터 항목
데이터는 어디에 있는가?	데이터 출처
데이터 품질은 충분한가?	활용 가능성
개인정보나 저작권 이슈는 없는가?	사용 가능성

데이터는 기능에서 나온다

데이터 요구사항은 막연히 많이 적는 것이 아닙니다.

기능과 연결되어야 합니다.

기능: 맞춤 추천

↓

필요 데이터: 사용자 조건, 추천 대상, 추천 기준

기능: 리뷰 요약

↓

필요 데이터: 리뷰 원문, 평점, 작성일

기능별 데이터 예시

기능	필요한 데이터
검색	이름, 카테고리, 태그, 설명
추천	사용자 조건, 추천 대상, 추천 기준
비교	항목별 수치, 가격, 특징, 장단점
요약	원문 텍스트, 작성자, 작성일
상담	FAQ, 정책, 지식 문서, 주의사항
알림	사용자 설정, 일정, 상태 변화

데이터의 종류

서비스 기획에서 자주 다루는 데이터는 크게 나눌 수 있습니다.

구분	설명	예시
사용자 데이터	개인화에 필요한 데이터	연령대, 선호도, 사용 이력
상품/대상 데이터	서비스가 제공하거나 추천하는 대상	상품명, 장소명, 콘텐츠명
행동 데이터	사용자의 이용 과정에서 생기는 데이터	클릭, 검색어, 구매, 조회
지식 데이터	설명과 판단의 근거가 되는 데이터	FAQ, 규정, 문서, 기준표
운영 데이터	서비스 운영과 품질 관리에 필요한 데이터	오류 로그, 문의, 만족도

정형 데이터와 비정형 데이터

구분	설명	예시
정형 데이터	표처럼 행과 열로 정리된 데이터	CSV, 엑셀, DB 테이블
비정형 데이터	정해진 표 구조가 없는 데이터	리뷰, 문서, 이미지, 음성
반정형 데이터	구조는 있지만 자유도가 있는 데이터	JSON, XML, API 응답

AI 서비스에서는 세 가지 데이터가 함께 사용되는 경우가 많습니다.

데이터 분류

[원본 슬라이드 이미지 - 참고용, 본 문서에는 미포함]
alt text

시간이 지날수록비정형 데이터에 대한 양이 늘어나고 있음

[원본 슬라이드 이미지 - 참고용, 본 문서에는 미포함]
alt text

데이터 항목 정의

데이터 요구사항은 “데이터가 필요하다”에서 끝나면 안 됩니다.
구체적인 항목까지 적어야 합니다.

나쁜 예	좋은 예
상품 데이터가 필요하다.	상품명, 브랜드, 카테고리, 가격, 설명, 이미지가 필요하다.
사용자 데이터가 필요하다.	연령대, 선호 조건, 관심사, 제외 조건이 필요하다.
리뷰 데이터가 필요하다.	리뷰 본문, 평점, 작성일, 상품 ID가 필요하다.

데이터 출처

데이터 출처는 데이터를 얻을 수 있는 위치입니다.
예시 - 내부 서비스 DB - 공공데이터 - API - CSV/엑셀 파일 - 사용자 입력 - 제휴사 제공 데이터 - 운영 중 쌓이는 로그 데이터

출처가 불명확한 데이터는 기획 단계에서 위험 요소가 됩니다.

공공데이터란?

공공데이터는 공공기관이 만들거나 관리하는 데이터입니다.
특징 - 출처가 비교적 명확하다. - 정책, 통계, 행정, 안전, 교통, 식품 등 다양한 분야에 존재한다. - CSV, 엑셀, API 형태로 제공될 수 있다. - 사용 조건과 갱신 주기를 확인해야 한다.

공공데이터를 볼 때 확인할 것

확인 항목	질문
제공 기관	누가 만든 데이터인가?
제공 항목	어떤 컬럼 또는 응답값이 있는가?
갱신 주기	얼마나 자주 업데이트되는가?
파일/API 형태	CSV인가, API인가, 문서인가?
이용 조건	상업적 활용이 가능한가? 출처 표기가 필요한가?
품질	빈값, 중복, 오래된 값이 많은가?

공공데이터 포털

[원본 슬라이드 이미지 - 참고용, 본 문서에는 미포함]
alt text

API란?

API는 다른 시스템의 데이터를 정해진 방식으로 요청하고 받는 통로입니다.



API는 실시간 또는 최신 데이터 연동이 필요할 때 자주 사용됩니다.

API 문서에서 볼 것

AI 기획자는 API 코드를 직접 작성하지 않더라도 문서의 핵심은 읽을 수 있어야 합니다.

항목	의미
API 이름	어떤 데이터를 제공하는가
요청 주소	데이터를 어디로 요청하는가
요청 파라미터	검색어, 날짜, 페이지 번호 등
응답 항목	실제로 받을 수 있는 데이터
인증 방식	API Key가 필요한가
호출 제한	하루 또는 초당 몇 번 호출 가능한가
오류 코드	실패 상황을 어떻게 알려주는가

[공공데이터 포털](#)

[원본 슬라이드 이미지 - 참고용, 본 문서에는 미포함]
alt text

CSV와 API 비교

구분	CSV/엑셀	API
사용 방식	파일을 내려받아 사용	요청할 때마다 데이터를 받음
장점	이해하기 쉽고 실습에 좋음	서비스 연동과 최신화에 유리
단점	최신 상태 유지가 어려움	인증, 호출 제한, 개발 연동 필요
기획 포인트	데이터 구조 파악에 좋음	실제 서비스 운영 가능성 검토에 좋음

데이터 활용 가능성 검토

데이터를 찾았다고 바로 사용할 수 있는 것은 아닙니다.

다음 기준으로 검토합니다.

기준	확인 질문
적합성	기능에 필요한 항목이 있는가?
신뢰성	출처가 명확하고 믿을 수 있는가?
최신성	최신 데이터인가? 갱신되는가?
완전성	누락, 빈값, 중복이 많지 않은가?
사용성	CSV, API 등 실제 활용 가능한 형태인가?

법적 이슈	저작권, 개인정보, 이용 조건 문제가 없는가?
-------	---------------------------

AI 기능에서 더 중요한 기준

AI 기능은 일반 기능보다 데이터 근거가 더 중요합니다.

기준	확인 질문
설명 가능성	AI가 결과의 이유를 설명할 수 있는가?
개인화 가능성	사용자 조건과 연결할 수 있는 항목이 있는가?
검색 가능성	RAG에서 검색할 문서나 텍스트가 있는가?
안전성	잘못된 답변이 사용자에게 피해를 줄 수 있는가?
제한 조건	답변하면 안 되는 범위가 있는가?

데이터 요구사항 문장 공식

데이터 요구사항은 다음 형태로 작성하면 명확합니다.

[기능명]을 제공하기 위해
[데이터 항목]이 필요하다.
해당 데이터는 [출처]에서 확보할 수 있으며,
[활용 방식]에 사용된다.

작성 예시

맞춤 추천 기능을 제공하기 위해 사용자 조건, 추천 대상 정보, 추천 기준 데이터가 필요하다.

해당 데이터는 사용자 입력, 상품 DB, 추천 기준표에서 확보할 수 있다.

이 데이터는 사용자의 조건에 맞는 후보를 좁히고 추천 이유를 설명하는 데 사용된다.

데이터 요구사항 표 예시

항목	내용
기능명	맞춤 추천
필요한 데이터	사용자 조건, 추천 대상, 추천 기준
데이터 출처	사용자 입력, 내부 DB, 공공데이터
데이터 형태	CSV, API, DB
활용 방식	조건에 맞는 후보 선별 및 추천 이유 생성
검토 사항	개인정보, 출처 신뢰도, 최신성

흔한 실수

- 기능만 정의하고 필요한 데이터를 적지 않는다.
- 데이터 출처를 “인터넷”처럼 모호하게 적는다.
- API가 있다는 이유만으로 바로 활용 가능하다고 판단한다.
- 개인정보가 필요한데 수집 목적과 범위를 정하지 않는다.
- AI가 답변할 근거 데이터 없이 챗봇 기능을 기획한다.

Day 04 — 가설수립 · KPI설계

자료 출처: 가설_수립_및_KPI_설계.md

가설 수립 및 KPI 설계

Pain Point를 찾았다면, 이제 “이렇게 해결하면 좋아질 것이다”라는 검증 가능한 가설로 바꾸고, 그 결과를 확인할 지표를 정해야 합니다.

오늘의 학습 목표

- 문제 정의를 검증 가능한 가설로 전환할 수 있다.
- 가설 수립 문장 공식을 활용해 가설을 작성할 수 있다.
- 성공 여부를 판단하는 KPI를 설계할 수 있다.
- AI 적용 가능성을 검토하는 기준을 적용할 수 있다.

지금까지의 흐름

문제 발견
↓
Pain Point 정의
↓
데이터 요구사항 분석
↓
가설 수립
↓
KPI 설정
↓
AI 적용 가능성 검토

Pain Point는 “무엇이 문제인가”였고, 가설은 “어떻게 해결하면 무엇이 좋아지는가”입니다.

가설이란?

가설은 문제를 해결하는 방법과 그 결과를 미리 예측한, 검증 가능한 문장입니다.

이 문제를

이렇게 해결하면

이런 결과가 나올 것이다

가설은 정답이 아니라 “확인해야 할 예측”입니다.

왜 가설이 필요한가?

아이디어만으로는 무엇을 확인해야 하는지 알 수 없습니다.

구분	의미	예시
아이디어	떠오른 생각	AI로 추천을 해주면 좋을 것 같다.
가설	검증 가능한 예측	맞춤 추천을 제공하면 선택 소요 시간이 30% 줄어든 것이다.
검증	가설이 맞았는지 확인	실제 소요 시간을 측정해 비교한다.

가설이 없으면 무엇을 측정해야 할지도 정할 수 없습니다.

가설의 조건

좋은 가설은 다음 조건을 만족합니다.

- 해결책과 기대 결과가 모두 담겨 있다.
- 결과를 숫자나 관찰로 확인할 수 있다.
- 대상 사용자와 상황이 구체적이다.
- 원인과 결과의 연결이 논리적이다.
- 참/거짓을 실제로 확인할 수 있다.

가설 문장 공식

가설은 다음 형태로 작성하면 명확해집니다.

[사용자]에게 [해결책]을 제공하면,
[지표]가 [방향/정도]로 변할 것이다.

가설 작성 예시

나쁜 예시

“AI 추천을 넣으면 좋아질 것이다.”

좋은 예시

“초보 사용자에게 조건 기반 맞춤 추천을 제공하면, 선택까지 걸리는 시간이 지금보다 30% 줄어들 것이다.”

Pain Point → 가설 전환 예시

Pain Point	가설
선택지가 많아 결정하기 어렵다.	맞춤 추천을 제공하면 평균 선택 시간이 줄어들 것이다.
리뷰가 많아 핵심을 파악하기 어렵다.	리뷰 요약을 제공하면 리뷰 확인 후 이탈률이 낮아질 것이다.
설명이 어려워 이해하기 힘들다.	쉬운 설명으로 바꿔주면 재질문(문의) 횟수가 줄어들 것이다.

하나의 Pain Point에서 여러 개의 가설이 나올 수 있습니다.

가설을 세울 때 흔한 실수

- 해결책만 있고 기대 결과가 없다. (예: “챗봇을 만들면 좋다.”)
- 기대 결과가 있지만 측정할 수 없다. (예: “더 편해질 것이다.”)
- 대상 사용자가 명확하지 않다.
- 한 가설에 너무 많은 기대를 담는다.

가설은 한 번에 하나씩, 검증 가능한 크기로 쪼개야 합니다.

KPI란?

KPI(Key Performance Indicator)는 가설이 맞았는지, 서비스가 목표대로 작동하는지 판단하는 정량적 지표입니다.

가설
↓
"무엇이 좋아지면 성공인가?"
↓
KPI (측정 가능한 숫자)

KPI가 필요한 이유

- 가설이 맞았는지 숫자로 확인할 수 있다.
- 기능 출시 전후를 비교할 수 있다.
- 팀 전체가 같은 기준으로 성공을 판단할 수 있다.
- 우선순위를 정할 근거가 생긴다.

좋은 KPI의 조건

좋은 KPI는 다음 조건을 만족합니다.

조건	의미
구체적	무엇을 어떻게 측정하는지 명확하다.
측정 가능	실제 데이터로 확인할 수 있다.
가설과 연결	가설의 기대 결과와 직접 연결된다.
기간 명시	언제까지, 얼마나 확인할지 정해져 있다.
통제 가능	우리 서비스의 개선으로 영향을 줄 수 있다.

KPI 문장 공식

KPI는 다음 형태로 작성하면 명확해집니다.

[기능/서비스]의 성공은
[지표]가 [기간] 동안
[목표 수치]를 달성했을 때로 판단한다.

KPI 작성 예시

맞춤 추천 기능의 성공은, 출시 후 4주 동안 평균 선택 소요 시간이 30% 감소했을 때로 판단한다.

리뷰 요약 기능의 성공은, 출시 후 4주 동안 상세페이지 이탈률이 15% 이상 낮아졌을 때로 판단한다.

KPI의 유형

구분	의미	예시
선행지표(Leading)	결과가 나오기 전에 먼저 움직이는 지표	기능 사용률, 클릭률
후행지표(Lagging)	최종 성과를 보여주는 지표	매출, 재구매율, 이탈률
사용성 지표	서비스를 얼마나 편하게 쓰는지	체류 시간, 완료율, 재사용률
AI 성능 지표	AI 기능 자체의 품질	정확도, 응답 시간, 만족도

후행지표만 보면 문제를 늦게 발견합니다. 선행지표와 함께 봐야 합니다.

AI 서비스에서 자주 쓰이는 KPI

영역	지표 예시
정확성	정답률, 정확도, 오답/오류율
응답 품질	사용자 만족도, 재질문 비율
성능	응답 시간, 처리 속도
사용 행태	기능 사용률, 재사용률, 완료율
비즈니스 성과	이탈률, 체류 시간, 전환율, 문의 감소율

모델 성능만 좋다고 서비스가 성공하는 것은 아닙니다. 비즈니스 지표와 함께 확인해야 합니다.

KPI 설계 시 흔한 실수

- 측정할 수 없는 지표를 세운다. (예: “사용자가 더 만족한다.”)
- 지표가 너무 많아 무엇이 중요한지 알 수 없다.
- 가설과 상관없는 지표를 KPI로 잡는다.
- 목표 수치와 기간이 없다.
- 모델 성능 지표만 확인하고 실제 사용자 반응은 확인하지 않는다.

지표는 최소한으로, 가설과 직접 연결된 것만 남깁니다.

AI 적용 가능성 검토란?

가설과 KPI가 정해져도, 그 해결책이 정말 AI로 구현해야 하는지는 별도로 검토해야 합니다.

가설: 이 해결책이면 좋아질 것이다
↓
"이 해결책은 AI가 꼭 필요한가?"
↓
AI 적용 가능성 검토

AI가 필요한지 먼저 판단한다

AI를 쓰는 것이 목적이 되어서는 안 됩니다.

상황	적합한 방식
규칙이 명확하고 조건이 단순하다	규칙 기반(Rule-based) 로직
조건에 따라 결과가 복잡하게 달라진다	AI/모델 기반
자연어 질문, 문서 요약, 패턴 예측이 필요하다	AI/모델 기반
정해진 값 조회, 계산, 단순 필터링이면 충분하다	규칙 기반 로직

“AI로 할 수 있다”와 “AI로 해야 한다”는 다릅니다.

AI 적용 가능성 검토 기준

기준	확인 질문
데이터 가용성	학습·판단에 필요한 데이터가 실제로 있는가?
문제 복잡도	규칙으로 정의하기 어려운 문제인가?
정확도 요구 수준	어느 정도 오류가 허용되는가?
기술 성숙도	현재 기술/모델로 구현 가능한 수준인가?
비용 대비 효과	개발·운영 비용보다 얻는 가치가 큰가?

리스크	오류가 사용자에게 주는 피해가 큰가?
설명 가능성	결과의 이유를 사용자에게 설명할 수 있는가?

정확도 요구 수준에 따른 판단

AI는 100% 정확하지 않습니다. 그래서 허용 오차를 먼저 정해야 합니다.

도메인	오류 허용도	판단
콘텐츠 추천	오류에 비교적 관대함	AI 적용에 적합
리뷰 요약	일부 오류는 허용 가능	AI 적용에 적합
의료·법률·금융 판단	오류 허용도가 매우 낮음	신중한 검토 또는 사람 확인 병행 필요

오류의 영향이 큰 영역일수록 AI 단독 판단보다 사람의 확인 절차를 함께 설계해야 합니다.

AI 적용 가능성 검토 예시

가설: 맞춤 추천을 제공하면 선택 소요 시간이 줄어든 것이다.

- 데이터 가용성: 사용자 조건, 상품 데이터 확보 가능 → 적합
- 문제 복잡도: 조건 조합이 많아 규칙만으로는 관리 어려움 → AI 적합
- 리스크: 잘못 추천해도 사용자가 다시 선택 가능 → 낮음
- 결론: AI 기반 추천으로 적용 가능성 있음

가설 · KPI · AI 적용 검토 한눈에 보기

항목	내용
Pain Point	선택지가 많아 결정하기 어렵다.
가설	맞춤 추천을 제공하면 선택 소요 시간이 30% 줄어든 것이다.
KPI	출시 후 4주 내 평균 선택 소요 시간 30% 감소
AI 적용 가능성	데이터 확보 가능, 규칙만으로 구현 어려움 → AI 적합

이 네 가지가 한 줄로 이어져야 다음 단계인 기능 정의와 개발이 흔들리지 않습니다.

흔한 실수

- 해결책만 정하고 검증할 가설을 세우지 않는다.
- 가설과 상관없는 KPI를 세운다.
- KPI에 목표 수치와 기간을 넣지 않는다.
- AI 적용 가능성을 검토하지 않고 바로 AI 기능으로 확정한다.
- 오류 허용 수준이 낮은 도메인에서 사람 확인 절차 없이 AI에만 맡긴다.

오늘 기억할 문장

가설은 검증할 대상이고, KPI는 그 검증 기준이며, AI 적용 가능성 검토는 그 해결책이 AI여야 하는 이유입니다.

이 세 가지가 갖춰지면, 다음 단계에서 어떤 기능을 어떻게 만들지 훨씬 명확해집니다.

Day 05 — 요구사항정의서

자료 출처: 요구사항정의서.md

요구사항정의서 (Requirements Specification)

참고: [요구사항 정의서 작성법과 양식 \(이런서 블로그\)](#)

요구사항정의서란?

요구사항정의서는 IT 프로젝트의 성공을 좌우하는 중요한 문서로, 프로젝트의 목표, 기능, 제약 사항 등을 명확하게 정의하여 전체적인 방향을 제시합니다.

- **요구사항(Requirement)** : 사용자·고객·프로젝트가 원하는 기능, 성능, 제약 조건
- **정의서(Specification)** : 그 요구사항을 명확하고 검증 가능하게 적어 둔 문서

사업 관리, 기획, 디자인, 개발, 테스터 등 **다양한 부서가 협업** 하여 의사소통을 원활히 할 수 있게 하며, **각 담당자별 역할을 명시** 하고 프로젝트 진행 차질을 방지하여 **성공적으로 완성** 되도록 돕는 문서입니다.

왜 작성해야 하는가?

요구사항 정의서를 제대로 작성하지 않으면 프로젝트 중도 포기, 계약 분쟁, 예산 초과, 일정 지연 등의 문제가 발생할 수 있습니다. 잘 작성된 요구사항 정의서는 프로젝트 성공 수행을 보장하는 필수 문서입니다.

두 가지 핵심 목적

목적	설명
1. 프로젝트 목표 정의	이해관계자에게 요구사항을 배포·공유하여 동일한 목표 범위와 목표의식 을 심어 줍니다.
2. 기준 문서 정의	완수해야 할 업무 범위 를 정의하는 기준 문서로, 협력 부서와 동일한 눈높이 를 설정하고 의사결정·판단의 근거 가 됩니다.

작성하지 않으면 생기는 문제

- **방향성 상실** : 요구사항이 불명확하거나 미정의되어 프로젝트 방향을 잃음
- **의견 불일치·갈등** : 이해관계자·담당자 간 인식 차이로 갈등 고조
- **일정 지연·비용 초과** : 수익성 저하로 이어짐
- **품질 저하·프로젝트 실패** : 잘못된 기능 구현 가능성 증가 → 결국 실패로 이어질 수 있음

어떻게 작성하는가?

4단계 작성 절차

단계	내용
1. 요구사항 수집	이해관계자와 인터뷰, 설문조사, 워크숍 등으로 수집. 각자의 요구와 기대를 명확히 이해하되, 프로젝트 성공에 필요한 요건만 도출하고, 프로젝트와의 관련도·중요성을 판단하며 수집합니다.
2. 요구사항 분석	수집된 요구사항에서 중복 제거, 우선순위 설정. 개인 취향이 아닌 프로젝트 핵심 요구사항 인지 면밀히 판단합니다.
3. 요구사항 그룹	기능적 요구사항/시스템이 수행해야 할 기능과 비기능적 요구사항(성능, 보안, 유지보수성)

3. 요구사항 구분	기능적 요구사항(시스템이 무엇을 할 기능)과 비기능적 요구사항(성능, 보안, 사용성, 운영 시 예상 이슈 등)으로 구분합니다.
4. 요구사항 정의서 작성	구분된 요구사항을 문서화합니다. 아래 다섯 가지를 유념해 작성합니다.

작성 시 반드시 포함할 다섯 가지

1. 프로젝트 인식

프로젝트명, 목적·배경·범위, 메뉴 구성, 뎁스(Depth) 등으로 프로젝트를 명확히 표현합니다.

[원본 슬라이드 이미지 - 참고용, 본 문서에는 미포함]
alt text

2 이해관계자 식별

요구사항 출처(누가), 수용 여부, 해당 요구로 협력·처리해야 할 이해관계자가 누구인지 식별합니다.

[원본 슬라이드 이미지 - 참고용, 본 문서에는 미포함]
alt text

3 기능적 요구사항

시스템이 수행해야 할 기능을 정의합니다. (예: 로그인, 데이터 저장, 보고서 생성) 기획·디자인·개발 업무의 실질적 범위입니다.

[원본 슬라이드 이미지 - 참고용, 본 문서에는 미포함]
alt text

4. 비기능적 요구사항

기능적 요구사항(Functional Requirements): 서비스가 무엇을 하는지(What)

비기능적 요구사항(Non-Functional Requirements): 서비스를 얼마나 잘 수행해야 하는지(How Well)

구분	기능적 요구사항	비기능적 요구사항
질문	무엇을 하는가?	어떻게 동작해야 하는가?
목적	기능 정의	품질 기준 정의
예시	회원가입, 로그인, 추천	응답속도, 보안, 안정성
테스트	기능 동작 여부	성능·품질 측정

5. 제약 사항 및 예상 결과

예산, 일정, 기술적 제한 등 프로젝트에 영향을 미치는 외부 요인을 정의합니다. 예상 결과가 좋지 않을 경우 대처방안을 사전에 준비합니다.

[원본 슬라이드 이미지 - 참고용, 본 문서에는 미포함]
alt text

작성 시 유의사항

- 명확한 표현**: “사용자는 로그인을 할 수 있어야 한다”보다 **“이메일과 비밀번호를 사용하여 로그인할 수 있어야 한다”**처럼 구체적으로. 정량적 기준 명시 (예: “빠르게” → “응답시간 2초 이내”). 동일 개념은 일관된 용어 사용.
- 일관된 형식**: 모든 요구사항을 동일한 템플릿에 기재. Depth, 설명, 우선순위, 상태 등을 템플릿화하고, 일련번호 형식의 인식 코드 부여. 글꼴, 폰트, 문단 스타일도 통일.
- 변경 기록·버전 관리**: 요구사항 변경 내용을 기록하고 버전 관리(예: v1.0, v1.1). 진행 상황·이슈를 추적 관리. 정기적 최신화 후 담당자 공유, 검토·승인 절차와 수정 권한 규정을 두는 것이 좋습니다.

요구사항 표현 방법

- **사용자 스토리** : “~로서, ~를 원해서, ~할 수 있어야 한다.” (애자일에서 자주 사용)
 - **shall / must** : “시스템은 ~해야 한다.” 형태로 필수 요구사항 명시
 - **번호 + 제목 + 설명**: **REQ-001** : 로그인 - 사용자는 이메일과 비밀번호로 로그인할 수 있다.
-

예제영상> 바이트코딩 직전까지의 기획 계획 과정

1. 문제 정의
2. AI로 시장조사 하는 법
3. 패턴 찾기
4. 핵심 가치 찾기
5. PRD 작성
6. 실패 가능성 묻기
7. 암묵적 가정 묻기
8. 마케팅 타겟 정하기
9. 예외 케이스

2단계. 데이터 & DB 설계 (day01 ~ day05)

Day 01 — 데이터개요

자료 출처: AI_서비스와_데이터의_이해.md

AI 서비스와 데이터의 이해

AI 기획자는 데이터를 깊게 분석하는 사람이 아니라, **AI가 믿고 사용할 수 있는 데이터인지 질문할 수 있는 사람**이어야 합니다.

학습 목표

- AI 서비스에서 데이터가 왜 중요한지 기획자 관점으로 설명할 수 있다.
- 정형 데이터와 비정형 데이터를 서비스 활용 관점에서 구분할 수 있다.
- 공공데이터와 API를 사용할 때 조심할 점을 이해할 수 있다.
- 데이터가 부족하거나 부정확할 때 AI 서비스에 어떤 문제가 생기는지 설명할 수 있다.

오늘의 핵심 질문

오늘은 어려운 데이터 분석 기법을 배우는 시간이 아닙니다.

AI 기획자가 최소한 다음 질문을 할 수 있도록 만드는 시간입니다.

- 이 AI는 무엇을 보고 답하는가?
- 그 데이터는 믿을 수 있는가?
- 사용자에게 보여 줘도 되는 데이터인가?
- 오래되거나 틀린 데이터는 없는가?
- 데이터가 없을 때 AI는 어떻게 행동해야 하는가?

AI 서비스는 데이터 위에서 움직인다

AI 서비스는 겉으로는 똑똑해 보입니다.

하지만 실제로는 데이터가 있어야 답하고, 추천하고, 비교하고, 설명할 수 있습니다.

기능	필요한 데이터
추천	사용자 조건, 추천 대상, 추천 기준
챗봇	답변할 문서, FAQ, 정책
검색	검색할 목록, 키워드, 설명
요약	요약할 글, 문서, 리뷰
알림	날짜, 상태, 사용자 설정

데이터는 AI의 재료다

요리를 할 때 재료가 나쁘면 결과도 좋아지기 어렵습니다.

AI도 마찬가지입니다.

- 데이터가 부족하면 답변이 부실해진다.
- 데이터가 오래되면 틀린 정보를 말할 수 있다.
- 데이터가 편향되어 있으면 결과도 한쪽으로 치우칠 수 있다.
- 데이터 출처가 불명확하면 사용자가 믿기 어렵다.

데이터는 AI의 기준이기도 하다

데이터는 AI가 참고하는 재료이면서, 동시에 판단 기준이 됩니다.

예를 들어,

- 어떤 상품을 추천할지 정하는 기준
- 어떤 답변을 하면 안 되는지 정하는 기준
- 어떤 사용자에게 다른 안내를 해야 하는지 정하는 기준
- 어떤 결과가 성공인지 판단하는 기준

AI 기획자는 “AI가 무엇을 기준으로 판단하는가”를 설명할 수 있어야 합니다.

데이터가 없으면 AI는 추측(거짓말)할 수 있다

AI는 모르는 내용을 만나도 자연스럽게 답하려고 할 수 있습니다.

그래서 데이터가 부족한 서비스에서는 다음 문제가 생깁니다.

- 없는 정보를 있는 것처럼 말한다.
- 사용자의 상황과 맞지 않는 추천을 한다.
- 최신 정책이나 가격을 틀리게 안내한다.
- 답변의 근거를 설명하지 못한다.
- 잘못된 답변을 나중에 고치기 어렵다.

기획자가 먼저 확인할 5가지

AI 기능을 기획할 때 데이터에 대해 가장 먼저 확인할 것은 다음입니다.

질문	의미
어디서 온 데이터인가?	출처 확인
언제 만들어진 데이터인가?	최신성 확인
무엇을 의미하는 데이터인가?	항목 의미 확인
사용해도 되는 데이터인가?	권한과 개인정보 확인
빠지거나 틀린 값은 없는가?	품질 확인

데이터가 많다고 좋은 것은 아니다

데이터는 많을수록 무조건 좋은 것이 아닙니다.

중요한 것은 양보다 **쓸모와 신뢰도**입니다.

좋지 않은 데이터	좋은 데이터
출처를 알 수 없다	출처가 명확하다
오래되었다	갱신일을 확인할 수 있다
중복이 많다	같은 값이 정리되어 있다
항목 뜻이 모호하다	항목 의미가 설명되어 있다
사용 조건이 불분명하다	사용 가능 범위가 확인되어 있다

원본 그대로는 부족할 수 있다

공공데이터, 엑셀 파일, 웹 문서, API 응답은 대부분 원본 상태로 제공됩니다.

하지만 서비스에서는 원본 그대로 쓰기 어려울 수 있습니다.

- 컬럼 이름이 이해하기 어렵다.
- 단위가 섞여 있다.
- 빈칸이 있다.

- 같은 의미가 다른 표현으로 적혀 있다.
- 서비스에 필요 없는 항목이 많다.

데이터를 찾는 것과 서비스에 맞게 쓰는 것은 다른 일입니다.

정형 데이터는 표처럼 정리된 데이터

정형 데이터는 행과 열로 정리된 데이터입니다.

엑셀, CSV, 데이터베이스 테이블이 대표적입니다.

이름	나이	가입일	구매횟수
김민수	28	2026-01-10	3
이서연	34	2026-02-15	1

정형 데이터는 무엇에 좋은가?

정형 데이터는 비교하고, 세고, 필터링하는 데 좋습니다.

- 조건에 맞는 사용자를 찾기
- 가격이 낮은 순서로 정렬하기
- 월별 가입자 수 계산하기
- 특정 카테고리만 보기
- 추천 후보 목록 만들기

표로 정리할 수 있는 기능은 정형 데이터와 잘 맞습니다.

정형 데이터에서 조심할 점

표로 되어 있다고 해서 항상 정확한 것은 아닙니다.

확인할 점	예시
단위	원인지 달러인지
날짜	작성일인지 수정일인지
빈값	모르는 값인지 해당 없음인지
중복	같은 사람이 두 번 들어갔는지
표현	남성/남/M 이 섞여 있지 않은지

비정형 데이터는 자유로운 형태의 데이터

비정형 데이터는 표로 딱 맞게 정리되지 않은 데이터입니다.

유형	예시
글	리뷰, 문의, 게시글, 문서
이미지	사진, 스캔 파일, 상품 이미지
음성	상담 녹음, 음성 메모
영상	강의 영상, 사용 장면

비정형 데이터는 왜 중요한가?

사용자의 실제 생각과 불편함은 비정형 데이터에 많이 들어 있습니다.

- 리뷰에는 사용자의 감정이 담긴다.
- 문의에는 반복되는 불편이 담긴다.

- 문서에는 서비스 정책과 지식이 담긴다.
- 이미지와 음성에는 텍스트로 설명하기 어려운 정보가 담긴다.

생성형 AI는 이런 데이터를 읽고 요약하거나 답변하는 데 자주 활용됩니다.

비정형 데이터에서 조심할 점

비정형 데이터는 풍부하지만 그대로 쓰기 어렵습니다.

- 문장이 너무 길거나 복잡할 수 있다.
- 중요한 내용과 잡음이 섞여 있다.
- 개인정보가 포함될 수 있다.
- 저작권 문제가 있을 수 있다.
- 오래된 문서와 최신 문서가 섞일 수 있다.

AI가 읽을 수 있다고 해서, 바로 서비스에 써도 된다는 뜻은 아닙니다.

정형과 비정형은 함께 쓰인다

실제 AI 서비스는 정형 데이터와 비정형 데이터를 함께 쓰는 경우가 많습니다.

상황	정형 데이터	비정형 데이터
상품 추천	가격, 카테고리, 평점	리뷰, 상세 설명
고객 상담	회원 등급, 주문 상태	문의 내용, FAQ
콘텐츠 검색	제목, 날짜, 태그	본문, 댓글

공공데이터는 좋은 출발점이다

공공데이터는 공공기관이 제공하는 데이터입니다.

AI 기획에서 공공데이터는 다음 이유로 유용합니다.

- 출처를 확인하기 쉽다.
- 정책, 통계, 기준 자료가 많다.
- 파일이나 API 형태로 제공되는 경우가 있다.
- 초기 서비스 아이디어를 검토할 때 도움이 된다.

공공데이터도 확인이 필요하다

공공데이터라고 해서 무조건 바로 쓸 수 있는 것은 아닙니다.

확인할 것	이유
제공 기관	신뢰할 수 있는 출처인지 확인
제공 항목	필요한 값이 있는지 확인
갱신일	오래된 데이터인지 확인
이용 조건	상업적 이용이나 출처 표기 확인
제공 방식	파일인지 API인지 확인

공공데이터에서 자주 생기는 문제

- 필요한 항목이 빠져 있다.
- 데이터가 너무 오래되었다.
- 파일 설명이 어렵다.
- 컬럼 이름이 이해하기 어렵다.
- API 사용 신청이 필요하다.
- 호출 횟수 제한이 있다.

공공데이터는 “찾았다”보다 “내 서비스에 맞게 쓸 수 있다”가 중요합니다.

API는 데이터를 주고받는 약속이다

API는 다른 시스템에서 데이터를 받아오거나 기능을 사용할 수 있게 해 주는 통로입니다.

쉽게 말하면,

```
우리 서비스가 요청한다
↓
다른 시스템이 데이터를 보내 준다
↓
우리 서비스가 그 데이터를 사용한다
```

API를 기획자가 알아야 하는 이유

AI 기획자가 API 코드를 직접 작성할 필요는 없습니다.

하지만 API가 있으면 다음을 판단할 수 있어야 합니다.

- 어떤 데이터를 받을 수 있는가?
- 검색 조건을 보낼 수 있는가?
- 응답이 느리면 어떻게 되는가?
- 하루에 몇 번이나 호출할 수 있는가?
- 실패했을 때 사용자에게 무엇을 보여 줄 것인가?

API에서 조심할 점

API는 연결만 되면 끝나는 것이 아닙니다.

문제	서비스 영향
응답이 느림	화면이 늦게 뜬다
호출 제한	사용자가 많으면 막힐 수 있다
인증키 필요	보안 관리가 필요하다
응답 형식 변경	기능이 갑자기 깨질 수 있다
외부 장애	우리 서비스도 영향을 받을 수 있다

개인정보는 적게 모을수록 안전하다

AI 서비스는 개인화를 위해 사용자 정보를 많이 요구하고 싶어질 수 있습니다.

하지만 일반적으로 개인정보는 적게 모을수록 안전합니다.

기획자는 다음을 먼저 생각해야 합니다.

- 꼭 필요한 정보인가?
- 덜 민감한 정보로 대신할 수 있는가?
- 사용자가 왜 필요한지 이해할 수 있는가?
- 언제 삭제할 것인가?

데이터 출처를 설명할 수 있어야 한다

AI가 어떤 답을 했을 때 사용자는 이렇게 물을 수 있습니다.

“이 답변은 어디를 보고 말한 건가요?”

기획자는 최소한 다음을 설명할 수 있어야 합니다.

- 어떤 데이터가 쓰였는가?

- 누가 제공한 데이터인가?
- 언제 기준의 데이터인가?
- 사용자가 확인할 수 있는 근거가 있는가?

오래된 데이터는 위험하다

AI가 틀리는 이유 중 하나는 오래된 데이터를 사용하기 때문입니다.

특히 다음 데이터는 최신성이 중요합니다.

- 가격
- 재고
- 정책
- 법규
- 일정
- 통계

“최신 데이터”라고 쓰기보다 “얼마나 자주 갱신되는가”를 확인해야 합니다.

잘못된 데이터는 잘못된 AI를 만든다

AI가 이상한 답을 했을 때 모델만 탓하면 안 됩니다.

데이터 자체가 문제일 수도 있습니다.

데이터 문제	AI 결과
누락	필요한 조건을 반영하지 못함
중복	같은 결과가 반복됨
오류	틀린 안내를 함
오래됨	현재와 다른 답을 함
편향	일부 사용자에게 불리한 결과를 냄

AI가 모를 때의 규칙도 필요하다

AI 서비스는 모든 질문에 답할 수 없습니다.

데이터가 없거나 근거가 부족한 경우의 규칙이 필요합니다.

- “확인된 정보가 없습니다”라고 말하기
- 상담원이나 담당 부서로 연결하기
- 사용자가 직접 확인할 링크 제공하기
- 추측하지 않도록 답변 범위 제한하기

기획자가 회의에서 던질 질문

AI 기능을 논의할 때 다음 질문을 던질 수 있으면 충분합니다.

- 이 기능에 꼭 필요한 데이터는 무엇인가?
- 그 데이터는 어디에서 가져오는가?
- 사용자에게 보여 줘도 되는 데이터인가?
- 데이터가 오래되면 어떻게 알 수 있는가?
- API가 실패하면 화면은 어떻게 되는가?
- AI가 모르는 질문을 받으면 어떻게 답해야 하는가?

너무 어렵게 생각하지 않기

AI 기획자가 처음부터 데이터 전문가가 될 필요는 없습니다.

다만 다음 감각은 꼭 필요합니다.

- 데이터가 없으면 AI도 제대로 일하기 어렵다.
- 데이터가 틀리면 AI도 틀릴 수 있다.
- 데이터 출처와 사용 조건은 확인해야 한다.
- 개인정보는 조심해서 다뤄야 한다.
- API는 편리하지만 장애와 제한이 있다.

핵심 정리

- AI 서비스에서 데이터는 답변과 판단의 근거다.
- 정형 데이터는 표처럼 정리되어 비교와 계산에 좋다.
- 비정형 데이터는 글, 이미지, 음성처럼 맥락이 풍부하다.
- 공공데이터는 유용하지만 갱신일, 항목, 이용 조건을 확인해야 한다.
- API는 데이터를 주고받는 통로지만 속도, 제한, 장애를 고려해야 한다.
- AI 기획자는 데이터를 깊게 분석하기보다, 위험한 지점을 질문할 수 있어야 한다.

오늘 기억할 문장

AI 기획자는 데이터를 직접 만드는 사람보다, AI가 믿고 쓸 수 있는 데이터인지 확인하는 사람에 가깝습니다.

Day 02 — 데이터전처리

자료 출처: 데이터_품질과_전처리_이해.md

데이터 품질과 전처리 이해

AI 기획자는 데이터를 직접 고치는 사람이 아니라, 데이터가 AI 서비스에 쓰일 만큼 믿을 수 있는 상태인지 판단할 수 있는 사람이어야 합니다.

학습 목표

- 데이터 품질이 AI 서비스에 왜 중요한지 설명할 수 있다.
- 결측치와 이상치가 무엇인지 이해할 수 있다.
- 전처리가 왜 필요한지 기획자 관점에서 설명할 수 있다.
- 데이터 품질 문제가 AI 성능과 사용자 경험에 미치는 영향을 말할 수 있다.

오늘의 핵심 질문

오늘은 어려운 통계나 코딩을 배우는 시간이 아닙니다.

AI 기획자가 다음 질문을 할 수 있도록 만드는 시간입니다.

- 이 데이터에 빠진 값은 없는가?
- 너무 이상한 값은 없는가?
- 같은 의미가 여러 방식으로 적혀 있지 않은가?
- AI가 이 데이터를 그대로 사용해도 되는가?
- 데이터 문제가 사용자에게 어떤 피해를 줄 수 있는가?

데이터 품질이란?

데이터 품질은 데이터가 서비스 목적에 맞게 믿고 사용할 수 있는 정도입니다.

좋은 데이터는 다음 조건을 만족합니다.

- 필요한 항목이 들어 있다.
- 값이 비어 있지 않다.
- 값이 너무 이상하지 않다.
- 같은 기준으로 입력되어 있다.
- 최신 상태에 가깝다.
- 출처와 의미를 설명할 수 있다.

AI 서비스에서 데이터 품질이 중요한 이유

AI는 데이터를 근거로 답하고 추천합니다.

데이터 품질이 낮으면 AI 기능도 불안정해집니다.

데이터 문제	AI 서비스 결과
값이 빠져 있음	조건을 제대로 반영하지 못함
값이 틀림	잘못된 답변이나 추천
값이 오래됨	현재와 맞지 않는 안내
표현이 제각각임	같은 대상을 다르게 인식
극단값이 섞임	평균, 추천, 예측이 왜곡

AI는 나쁜 데이터를 자동으로 고치지 않는다

AI가 똑똑해 보여도 데이터 문제를 항상 알아서 해결하지는 못합니다.

예를 들어,

- 나이가 비어 있는 사용자를 어떻게 처리할지
- 가격이 0원인 상품을 정상으로 볼지
- 리뷰 점수가 5점 만점인데 99점이 들어온 경우
- 같은 지역이 “서울”, “서울시”, “Seoul”로 섞인 경우

이런 문제는 AI 기능 이전에 정리 기준이 필요합니다.

전처리란?

전처리는 데이터를 AI나 서비스가 사용할 수 있도록 미리 정리하는 과정입니다.

쉽게 말하면,

```
원본 데이터
↓
틀린 값, 빈 값, 이상한 값 확인
↓
기준에 맞게 정리
↓
AI 서비스에 사용할 수 있는 데이터
```

전처리를 왜 해야 하는가?

전처리는 데이터를 예쁘게 만드는 작업이 아닙니다.

AI가 잘못 판단하지 않도록 데이터를 서비스 기준에 맞추는 작업입니다.

- 빠진 값을 확인한다.
- 이상한 값을 찾는다.
- 단위와 표현을 맞춘다.
- 중복을 정리한다.
- 필요 없는 항목을 제거한다.
- 개인정보나 민감정보를 조심한다.

기획자가 전처리를 알아야 하는 이유

AI 기획자가 직접 전처리를 수행하지 않아도, 전처리의 필요성은 이해해야 합니다.

기획 상황	필요한 판단
기능 범위 정하기	지금 데이터로 가능한 기능인지 판단
일정 산정	데이터 정리에 시간이 필요한지 판단
요구사항 작성	어떤 품질 기준이 필요한지 작성
테스트 설계	어떤 데이터 문제가 발생할 수 있는지 확인
사용자 안내	결과가 제한될 수 있는 상황 설명

결측치란?

결측치는 데이터에서 값이 비어 있는 상태를 말합니다.

예를 들어,

이름	나이	지역	관심사
김민수	28	서울	운동
이서연		부산	여행
박지훈	35		음악

나이와 지역처럼 비어 있는 값이 결측치입니다.

결측치는 왜 생길까?

결측치는 여러 이유로 생깁니다.

- 사용자가 입력하지 않았다.
- 수집 과정에서 누락되었다.
- 원래 해당 없는 값이다.
- 시스템 오류로 저장되지 않았다.
- 개인정보라 일부러 수집하지 않았다.
- 아직 확인되지 않은 정보다.

비어 있다고 해서 모두 같은 의미는 아닙니다.

결측치에서 가장 중요한 질문

기획자는 먼저 결측치의 의미를 물어야 합니다.

질문	이유
왜 비어 있는가?	실수인지, 의도인지 구분
꼭 필요한 값인가?	기능 동작에 필수인지 판단
다른 값으로 대신할 수 있는가?	대체 가능성 확인
사용자에게 다시 물어봐야 하는가?	입력 흐름 개선
비워 둔 채 사용해도 되는가?	서비스 위험 판단

결측치를 무시하면 생기는 문제

결측치를 그냥 두면 AI 결과가 흔들릴 수 있습니다.

- 개인화 추천이 부정확해진다.
- 특정 사용자가 분석에서 제외된다.
- 평균이나 비율이 왜곡된다.
- AI가 부족한 정보를 추측할 수 있다.
- 사용자에게 엉뚱한 질문이나 답변을 할 수 있다.

결측치 처리 방향

결측치를 처리하는 방법은 하나가 아닙니다.

방법	의미	기획자가 확인할 점
다시 입력 받기	사용자에게 값을 요청	사용자 부담이 커지지 않는가
제외하기	해당 데이터를 사용하지 않음	중요한 사용자가 빠지지 않는가
대체하기	평균값, 기본값 등으로 채움	왜곡이 생기지 않는가
모름으로 표시	알 수 없음 상태로 유지	AI가 추측하지 않게 할 수 있는가

이상치란?

이상치는 일반적인 범위에서 크게 벗어난 값입니다.

예를 들어,

항목	일반적인 값	이상해 보이는 값
나이	20, 35, 48	999
가격	10,000원	-5,000원
평점	1~5점	99점

구매수량	1~10개	10,000개
------	-------	---------

이상치는 무조건 틀린 값일까?

이상치가 항상 오류는 아닙니다.

진짜 특별한 사례일 수도 있습니다.

가능성	예시
입력 오류	나이에 999 입력
단위 오류	10kg을 10g처럼 입력
시스템 오류	가격이 음수로 저장
진짜 특이 사례	매우 큰 주문량
이벤트 효과	할인 행사로 갑자기 구매 증가

이상치는 지우기 전에 이유를 확인해야 합니다.

이상치를 무시하면 생기는 문제

이상치가 섞이면 AI 판단이 왜곡될 수 있습니다.

- 평균값이 크게 흔들린다.
- 추천 기준이 이상해진다.
- 인기 상품이나 주요 사용자를 잘못 판단한다.
- 예측 결과가 과장될 수 있다.
- 운영자가 문제를 늦게 발견할 수 있다.

이상치에서 기획자가 물어볼 질문

질문	의미
정상 범위는 어디까지인가?	서비스 기준 확인
이 값은 실제 가능한가?	현실성 확인
입력 오류일 가능성이 있는가?	데이터 수집 방식 확인
그대로 두면 AI 결과에 영향이 큰가?	위험도 판단
사용자에게 보여도 되는 값인가?	UX와 신뢰도 확인

전처리란 기준을 정하는 일이다

전처리는 단순히 데이터를 삭제하거나 수정하는 일이 아닙니다.

기획 관점에서는 어떤 값을 정상으로 볼 것인가를 정하는 일입니다.

예를 들어,

- 나이는 0세 이상 120세 이하만 허용한다.
- 평점은 1점부터 5점까지만 허용한다.
- 가격은 0보다 커야 한다.
- 지역명은 정해진 목록에서 선택하게 한다.
- 필수값이 비어 있으면 다음 단계로 진행하지 않는다.

데이터 품질 기준 예시

AI 기능을 만들기 전, 간단한 품질 기준을 정해 둘 수 있습니다.

항목	품질 기준 예시
피수가	사용자의 사قم면은 비어 있으면 안 됨

데이터	사용자 ID, 상품 ID, 리뷰 텍스트 등
범위	나이는 0~120 사이
형식	날짜는 YYYY-MM-DD 형식
중복	같은 상품 ID는 한 번만 존재
단위	가격은 원화 기준
최신성	최근 1년 데이터 사용

데이터 품질이 AI 성능에 미치는 영향

데이터 품질은 AI의 정확도뿐 아니라 사용자 경험에도 영향을 줍니다.

품질 문제	AI 성능 영향	사용자 경험 영향
결측치	조건 반영 실패	맞지 않는 추천
이상치	판단 기준 왜곡	이상한 결과 노출
중복	특정 결과 과대표현	같은 답변 반복
오래된 데이터	최신 상황 반영 실패	신뢰도 하락
표현 불일치	같은 대상을 다르게 인식	검색 결과 누락

추천 AI에서 생기는 문제

추천 AI는 데이터 품질 영향을 크게 받습니다.

예를 들어,

- 사용자 선호 정보가 비어 있으면 추천이 일반적이 된다.
- 상품 카테고리가 틀리면 엉뚱한 상품이 추천된다.
- 평점 데이터에 이상치가 있으면 인기 상품 판단이 왜곡된다.
- 오래된 구매 이력만 있으면 현재 관심사를 반영하지 못한다.

챗봇 AI에서 생기는 문제

챗봇은 답변 근거 데이터의 품질이 중요합니다.

- FAQ가 오래되면 틀린 정책을 안내한다.
- 문서에 빈 내용이 많으면 답변이 부실해진다.
- 같은 질문에 대한 답이 여러 문서에서 다르면 답변이 흔들린다.
- 출처가 불분명하면 사용자가 답변을 믿기 어렵다.

검색 AI에서 생기는 문제

검색 기능도 데이터 품질의 영향을 받습니다.

- 제목이나 키워드가 비어 있으면 검색되지 않는다.
- 같은 단어가 여러 표현으로 섞이면 결과가 빠진다.
- 잘못된 태그가 있으면 엉뚱한 결과가 나온다.
- 오래된 문서가 먼저 나오면 사용자가 혼란스러워진다.

데이터 품질은 성능만의 문제가 아니다

데이터 품질 문제는 단순히 “AI가 조금 틀리는 문제”가 아닙니다.

서비스 전체의 신뢰와 연결됩니다.

- 사용자가 AI 답변을 믿지 않게 된다.
- CS 문의가 늘어난다.
- 잘못된 정보로 피해가 생길 수 있다.
- 운영자가 결과를 설명하기 어려워진다.
- 서비스 개선 방향을 잘못 판단할 수 있다.

AI가 추측하지 않게 하기

데이터가 부족할 때 AI가 추측하지 않도록 규칙을 정해야 합니다.

- 필요한 데이터가 없으면 “확인할 수 없습니다”라고 답한다.
- 오래된 정보는 최신 확인이 필요하다고 안내한다.
- 근거가 없는 추천은 하지 않는다.
- 사용자가 직접 입력해야 하는 정보는 다시 요청한다.
- 민감한 판단은 담당자 확인이 필요하다고 안내한다.

기획자가 회의에서 던질 질문

데이터 품질을 논의할 때 다음 질문을 던질 수 있으면 충분합니다.

- 이 데이터에서 꼭 필요한 값은 무엇인가?
- 비어 있으면 안 되는 항목은 무엇인가?
- 정상 범위는 어디까지인가?
- 이상한 값이 들어오면 어떻게 처리할 것인가?
- 오래된 데이터는 어떻게 구분할 것인가?
- AI가 근거 없는 답을 하지 않게 하려면 어떤 규칙이 필요한가?

너무 어렵게 생각하지 않기

AI 기획자가 전처리 코드를 직접 작성할 필요는 없습니다.

하지만 다음 감각은 꼭 필요합니다.

- 빈값은 AI 판단을 약하게 만든다.
- 이상한 값은 AI 판단을 왜곡한다.
- 같은 표현은 같은 방식으로 정리되어야 한다.
- 데이터 품질 기준은 기능 요구사항과 함께 정해야 한다.
- 데이터 문제가 사용자 경험 문제로 이어질 수 있다.

핵심 정리

- 데이터 품질은 AI 서비스의 신뢰도와 직결된다.
- 결측치는 값이 비어 있는 상태이며, 이유를 먼저 확인해야 한다.
- 이상치는 일반적인 범위에서 벗어난 값이며, 오류인지 특이 사례인지 구분해야 한다.
- 전처리는 AI가 데이터를 안정적으로 쓰도록 정리하는 과정이다.
- 데이터 품질이 낮으면 AI 성능뿐 아니라 사용자 경험도 나빠진다.
- AI 기획자는 데이터 품질 문제를 발견하고 질문할 수 있어야 한다.

오늘 기억할 문장

좋은 AI는 좋은 모델만으로 만들어지지 않습니다.

AI가 믿고 사용할 수 있는 데이터가 있어야 좋은 서비스가 됩니다.

Day 03 — 데이터정의서

자료 출처: 데이터_정의서.md

데이터 정의서 작성

데이터 정의서는 데이터를 개발자만 이해하는 표로 남기는 문서가 아닙니다. AI 기획자가 서비스 기능에 필요한 데이터를 누구나 같은 의미로 이해하도록 정리하는 문서입니다.

학습 목표

- 데이터 정의서가 왜 필요한지 설명할 수 있다.
- 데이터 식별이 무엇인지 이해할 수 있다.
- 데이터 속성을 어떤 기준으로 정의하는지 설명할 수 있다.
- 데이터 출처와 활용 목적을 데이터 정의서에 적는 이유를 이해할 수 있다.

오늘의 핵심 질문

오늘은 데이터베이스 설계를 깊게 배우는 시간이 아닙니다.

AI 기획자가 다음 질문에 답할 수 있도록 만드는 시간입니다.

- 우리 서비스에는 어떤 데이터가 필요한가?
- 각 데이터는 무엇을 의미하는가?
- 데이터 항목은 어떤 형식으로 관리해야 하는가?
- 이 데이터는 어디에서 가져오는가?
- 이 데이터는 어떤 기능에 사용되는가?

데이터 정의서란?

데이터 정의서는 서비스에서 사용할 데이터를 정리한 문서입니다.

쉽게 말하면,

```
데이터 이름
↓
데이터가 의미하는 것
↓
데이터 항목
↓
데이터 형식
↓
출처와 활용 목적
```

을 정리한 문서입니다.

왜 데이터 정의서가 필요한가?

같은 단어도 사람마다 다르게 이해할 수 있습니다.

예를 들어,

- “회원”은 가입만 한 사람인가, 실제 이용한 사람인가?
- “최근 구매”는 7일 이내인가, 30일 이내인가?
- “평점”은 5점 만점인가, 10점 만점인가?
- “상품명”은 화면에 보이는 이름인가, 내부 관리 이름인가?

데이터 정의서는 이런 해석 차이를 줄입니다.

AI 기획자에게 데이터 정의서가 중요한 이유

AI 서비스는 데이터 의미가 조금만 달라도 결과가 달라질 수 있습니다.

상황	데이터 정의서가 필요한 이유
추천 기능	어떤 조건으로 추천하는지 명확히 해야 한다
챗봇 기능	어떤 문서를 근거로 답하는지 알아야 한다
검색 기능	어떤 항목을 검색 대상으로 볼지 정해야 한다
개인화 기능	어떤 사용자 정보를 사용할지 정해야 한다
관리자 화면	어떤 값을 수정하고 관리할지 정해야 한다

데이터 정의서에 들어가는 기본 내용

데이터 정의서는 보통 다음 내용을 포함합니다.

구분	의미
데이터명	어떤 데이터인지 나타내는 이름
설명	데이터가 무엇을 의미하는지
주요 항목	데이터 안에 들어가는 세부 값
형식	문자, 숫자, 날짜 등 값의 형태
필수 여부	반드시 있어야 하는 값인지
출처	어디서 가져오는 데이터인지
활용 목적	어떤 기능에 쓰이는지

데이터 식별이란?

데이터 식별은 서비스에 필요한 데이터를 찾아 이름 붙이는 과정입니다.

쉽게 말하면,

“우리 서비스에는 어떤 데이터 묶음이 필요한가?”

를 정하는 일입니다.

예를 들어,

- 사용자 데이터
- 상품 데이터
- 주문 데이터
- 리뷰 데이터
- 문의 데이터
- 문서 데이터

처럼 데이터 묶음을 구분합니다.

데이터 식별은 기능에서 출발한다

데이터는 막연히 모으는 것이 아닙니다.

기능을 기준으로 필요한 데이터를 찾습니다.

기능	식별할 수 있는 데이터
회원가입	사용자 데이터
상품 검색	상품 데이터, 카테고리 데이터
맞춤 추천	사용자 조건 데이터, 추천 대상 데이터

리뷰 요약	리뷰 데이터
문의 응답	FAQ 데이터, 문의 데이터

데이터 식별 시 조심할 점

데이터 이름을 너무 넓게 잡으면 문서가 모호해집니다.

모호한 이름	더 나은 이름
정보 데이터	사용자 기본 정보
상품 관련 데이터	상품 기본 정보
AI 데이터	추천 기준 데이터
상담 데이터	고객 문의 데이터
문서 데이터	FAQ 문서 데이터

이름만 보고도 어떤 데이터인지 어느 정도 알 수 있어야 합니다.

데이터와 데이터 항목은 다르다

데이터는 큰 묶음이고, 데이터 항목은 그 안의 세부 값입니다.

데이터	데이터 항목
사용자 데이터	사용자ID, 이름, 연령대, 지역
상품 데이터	상품ID, 상품명, 가격, 카테고리
리뷰 데이터	리뷰ID, 작성자, 평점, 리뷰 내용
문의 데이터	문의ID, 제목, 본문, 처리 상태

데이터 정의서에서는 이 둘을 구분해야 합니다.

데이터 속성이란?

데이터 속성은 데이터 항목의 자세한 설명입니다.

예를 들어 **가격**이라는 항목이 있다면 다음을 정의해야 합니다.

- 어떤 의미의 가격인가?
- 숫자인가, 문자인가?
- 단위는 원인가?
- 반드시 있어야 하는가?
- 0원이 가능한가?
- 누가 입력하거나 수정하는가?

데이터 속성에 포함할 내용

속성	설명
항목명	데이터 항목의 이름
설명	항목이 의미하는 내용
데이터 형식	문자, 숫자, 날짜 등
예시 값	실제 들어갈 수 있는 값
필수 여부	비어 있어도 되는지
허용 범위	값이 가질 수 있는 범위
비고	주의할 점이나 추가 설명

데이터 형식 정의

데이터 형식은 값의 형태를 정하는 것입니다.

형식	예시
문자	이름, 상품명, 설명
숫자	가격, 수량, 점수
날짜	가입일, 주문일, 수정일
선택값	성별, 상태, 카테고리
참/거짓	수신 동의 여부, 활성 여부

형식을 정해 두면 입력 오류를 줄일 수 있습니다.

필수 여부 정의

모든 데이터 항목이 반드시 필요한 것은 아닙니다.

항목	필수 여부 판단
사용자ID	사용자를 구분해야 하므로 필수
이름	서비스에 따라 필수 또는 선택
휴대폰 번호	꼭 필요한 경우에만 수집
관심사	개인화 기능에는 중요하지만 선택 가능
프로필 이미지	없어도 서비스 이용 가능

필수값이 많을수록 사용자 입력 부담도 커집니다.

허용 범위 정의

값이 들어올 수 있는 범위를 정하면 이상한 데이터를 줄일 수 있습니다.

예를 들어,

항목	허용 범위 예시
나이	0~120
평점	1~5
가격	0보다 큰 숫자
주문 수량	1개 이상
상태	접수, 처리중, 완료 중 하나

예시 값이 중요한 이유

데이터 정의서에는 예시 값을 함께 적는 것이 좋습니다.

예시가 있으면 문서를 읽는 사람이 더 빨리 이해할 수 있습니다.

항목명	설명	예시 값
사용자ID	사용자를 구분하는 고유값	U001
가입일	사용자가 가입한 날짜	2026-07-11
상태	현재 처리 상태	처리중
평점	사용자가 남긴 점수	4.5

데이터 출처란?

데이터 출처는 데이터가 어디에서 오는지를 의미합니다.

대표적인 출처는 다음과 같습니다.

- 사용자 입력
- 내부 서비스 DB
- 관리자 등록
- 공공데이터
- 외부 API
- 제휴사 제공 파일
- 운영 중 쌓이는 로그

출처를 적어야 하는 이유

출처가 없으면 데이터를 믿기 어렵습니다.

이유	설명
신뢰도 확인	어디서 온 데이터인지 알아야 한다
최신성 확인	언제 갱신되는지 확인할 수 있다
오류 추적	문제가 생겼을 때 원인을 찾기 쉽다
권한 확인	사용해도 되는 데이터인지 확인한다
AI 설명	답변 근거를 설명할 수 있다

활용 목적이란?

활용 목적은 그 데이터가 어떤 기능에 쓰이는지를 적는 것입니다.

예를 들어,

데이터	활용 목적
사용자 관심사	맞춤 추천에 사용
상품 카테고리	검색과 필터에 사용
리뷰 내용	리뷰 요약과 감성 파악에 사용
FAQ 문서	챗봇 답변 근거로 사용
주문 상태	배송 안내에 사용

활용 목적을 적어야 하는 이유

데이터는 목적 없이 모으면 안 됩니다.

기획자는 데이터를 수집할 때 다음을 설명할 수 있어야 합니다.

- 왜 이 데이터가 필요한가?
- 어떤 기능에 쓰이는가?
- 사용자에게 어떤 가치를 주는가?
- 꼭 수집해야 하는가?
- 덜 민감한 데이터로 대체할 수 있는가?

데이터 정의서 예시 구조

데이터 정의서는 다음과 같은 표로 작성할 수 있습니다.

데이터명	항목명	설명	형식	필수	예시	출처	활용 목적
사용자 데이터	사용자ID	사용자를 구분하는 값	문자	필수	U001	회원가입	사용자 식별
사용자 데이터	관심사	사용자가 선택한 관심사	문자	선택	영화	사용자 입력	맞춤 추천

사용자 데이터	편집사	사용자가 선택한 관심 분야	문서	인백	여백	사용자 입력	덧셈 수인
상품 데이터	가격	상품 판매 가격	숫자	필수	15000	내부 DB	검색/비교

좋은 데이터 정의서의 특징

좋은 데이터 정의서는 읽는 사람이 같은 의미로 이해할 수 있습니다.

- 데이터 이름이 명확하다.
- 항목 설명이 짧고 구체적이다.
- 형식과 예시 값이 있다.
- 필수 여부가 정해져 있다.
- 출처가 적혀 있다.
- 활용 목적이 기능과 연결되어 있다.
- 개인정보 여부를 확인할 수 있다.

좋지 않은 데이터 정의서의 특징

다음과 같은 정의서는 나중에 혼란을 만듭니다.

- 데이터명이 너무 넓다.
- 항목 설명이 없다.
- 예시 값이 없다.
- 필수값인지 선택값인지 모른다.
- 데이터 출처가 없다.
- 어떤 기능에 쓰는지 적혀 있지 않다.
- 같은 의미의 항목이 여러 이름으로 반복된다.

AI 서비스에서 특히 조심할 점

AI 서비스에서는 데이터 정의가 더 중요합니다.

왜냐하면 AI가 데이터를 보고 답하거나 추천하기 때문입니다.

조심할 점	이유
항목 의미가 모호함	AI가 잘못 해석할 수 있다
출처가 불명확함	답변 근거를 설명하기 어렵다
필수값이 비어 있음	개인화나 추천이 약해진다
오래된 데이터	틀린 안내가 나올 수 있다
민감한 정보	개인정보 문제가 생길 수 있다

기획자가 회의에서 던질 질문

데이터 정의서를 작성할 때 다음 질문을 던지면 좋습니다.

- 이 데이터는 어떤 기능 때문에 필요한가?
- 항목 이름만 보고 의미를 알 수 있는가?
- 값의 형식과 예시가 있는가?
- 비어 있어도 되는 값인가?
- 어디에서 가져오는 데이터인가?
- 누가 입력하거나 관리하는가?
- 사용해도 되는 데이터인가?

너무 어렵게 생각하지 않기

AI 기획자가 데이터베이스 전문가가 될 필요는 없습니다.

다만 다음 정도는 설명할 수 있어야 합니다.

- 어떤 데이터가 필요한지

- 각 데이터가 무엇을 의미하는지
- 어떤 항목이 꼭 필요한지
- 어디에서 가져오는 데이터인지
- 어떤 기능에 쓰이는지

핵심 정리

- 데이터 정의서는 데이터를 같은 의미로 이해하기 위한 문서다.
- 데이터 식별은 필요한 데이터 묶음을 찾고 이름 붙이는 일이다.
- 데이터 속성 정의는 항목의 의미, 형식, 필수 여부, 예시를 정하는 일이다.
- 데이터 출처는 신뢰도와 책임을 확인하기 위해 필요하다.
- 활용 목적은 데이터가 어떤 기능에 쓰이는지 설명한다.
- AI 기획자는 데이터를 깊게 설계하기보다, 데이터 의미와 사용 이유를 명확히 설명할 수 있어야 한다.

오늘 기억할 문장

데이터 정의서는 개발자를 위한 표가 아니라, 기획자와 개발자, 디자이너, 운영자가 같은 데이터를 같은 의미로 이해하게 만드는 약속입니다.

Day 04 — 데이터모델링

자료 출처: 1_DBMS_개요.pdf, 2_데이터_모델링.pdf, 3_데이터베이스_정규화.pdf

DBMS 개요

데이터(Data)란?

데이터의 종류는 일반적으로 정형 데이터, 반정형 데이터, 비정형 데이터의 3가지로 나눌 수 있다.

스키마(schema): 데이터의 구조와 제약 조건에 대한 것들을 정의한 것.

시간이 지날수록 비정형 데이터의 비중이 정형 데이터보다 훨씬 빠르게 늘고 있다(2008~2015년 통계 기준, 비정형 데이터가 정형 데이터 대비 압도적으로 큰 폭으로 증가).

- **정형 데이터(Structured Data):** 고정된 필드(스키마를 철저히 따름)에 저장된 데이터. 예: 관계형 데이터베이스, 스프레드시트(엑셀) 등
- **반정형 데이터(Semi-Structured Data):** 고정된 필드에 저장되어 있지는 않지만, 메타데이터나 스키마 등을 포함하는 데이터. 예: XML, HTML, JSON, 이메일 등
- **비정형 데이터(Unstructured Data):** 고정된 필드에 저장되어 있지 않은 데이터. 예: 텍스트, 이미지, 동영상, 음성 데이터 등

데이터베이스(DB; Database)

데이터베이스를 한 마디로 정의하면 '데이터의 집합'이다. 일상생활 대부분의 정보가 데이터베이스에 저장되고 관리된다(카카오톡 메시지, 인스타그램 사진, 교통카드 이용 내역, 카페 결제 내역 등).

데이터베이스의 특징

- **실시간 접근성(real time accessibility):** 사용자의 질의에 대하여 즉시 처리하여 응답
- **계속적인 진화(continuous evolution):** 삽입, 삭제, 갱신을 통하여 항상 최근의 정확한 데이터를 동적으로 유지
- **동시 공유(concurrent sharing):** 여러 사용자가 동시에 원하는 데이터를 공유
- **내용에 의한 참조(content reference):** 튜플의 주소나 위치가 아닌 사용자가 요구하는 데이터 내용에 따라 참조
- **데이터 논리적 독립성(independence):** 응용프로그램과 데이터베이스를 독립시킴으로써 데이터 논리적 구조가 변경되어도 응용프로그램은 변경되지 않음

데이터베이스 관리 시스템(DBMS; Database Management System)

DBMS란 데이터베이스를 조작하는 별도의 소프트웨어이다. DBMS를 통해 응용 프로그램들이 데이터베이스를 공유하고 사용할 수 있는 환경을 제공한다.

DBMS의 기능

- **정의:** 데이터에 대한 형식, 구조, 제약조건들을 명세
- **구축:** DBMS가 관리하는 기억 장치에 데이터를 저장
- **조작:** 특정 데이터 검색을 위한 질의, 데이터베이스의 갱신, 보고서 생성 등
- **공유:** 여러 사용자와 프로그램이 동시에 접근하도록 함
- **보호:** 하드웨어·소프트웨어 오작동이나 권한 없는 악의적 접근으로부터 보호
- **유지보수:** 시간이 지남에 따라 변화하는 요구사항을 반영

DBMS의 장점(Advantages of DBMS): Simplicity, Data Integration, Data Migration, Data Access, Decision Making, Backup & Recovery, Data Sharing, Data Integrity, Data Security, Data Privacy, Data Concurrency, Data Redundancy 방지, Data Inconsistency 방지, Low Maintenance, Multi-Access Support, End-User Productivity

DBMS 유형

Relational DBMS (RDBMS)

관계형 데이터베이스 관리 시스템(RDBMS)은 RDB(관계형 데이터베이스)를 관리하는 시스템이다. RDB는 테이블, 행, 열의 정보를 구조화하는 방식으로, 테이블을 조인하여 정보 간 관계·링크를 설정할 수 있어 여러 데이터 포인트 간의 관계를 쉽게 이해하고 정보를 얻을 수 있다.

- RDBMS 종류: Oracle, MySQL, PostgreSQL 등
- 예: 주문목록 테이블(주문ID, 고객ID, 상품ID, 주문시간, 상품명, 가격, 고객명, 전화번호, 주소지)을 정규화하면 → 고객 테이블(고객ID, 고객명, 전화번호, 주소지) + 상품 테이블(상품ID, 상품명, 가격) + 주문목록 테이블(주문ID, 고객ID, 상품ID, 주문시간)로 분리

NoSQL DBMS

NoSQL은 비관계형 데이터베이스를 지칭한다. 관계형 데이터 모델을 지양하며, 대량의 분산된 데이터를 저장·조회하는 데 특화되었고, 스키마 없이 사용 가능하거나 느슨한 스키마를 제공하는 저장소를 말한다.

- NoSQL DBMS 종류: MongoDB, Hbase, DynamoDB 등

- NoSQL 저장 방식: Column-Family, Graph, Document, Key-Value

RDB vs NoSQL DB 비교

구분	SQL(관계형)	NoSQL(비관계형)
구조	구조화된 쿼리 언어, 미리 정의된 스키마	비정형 데이터에 대한 동적 스키마
확장	수직 확장 가능(scale up)	수평 확장 가능(scale out)
저장 방식	테이블 기반	Document, 키-값, 그래프 또는 와이드 컬럼 저장소
특성	다중 레코드 트랜잭션에 좋다	문서나 JSON 같은 비정형 데이터에 좋다
트랜잭션	다중 레코드 ACID 트랜잭션 지원	다중 레코드 ACID 트랜잭션 미지원(MongoDB 등 일부는 지원)

SQL(Structured Query Language)

데이터베이스에 접근하고 조작하는 언어이며 세 가지로 구분된다.

- **DDL(Data Definition Language)**: 데이터베이스와 테이블을 정의, 수정, 삭제하는 구문
- **DML(Data Manipulation Language)**: 테이블의 데이터를 삽입, 조회, 수정, 삭제하는 구문
- **DCL(Data Control Language)**: 데이터의 보안, 무결성, 회복 등을 제어하는 구문

대표 RDBMS(MySQL, Oracle, PostgreSQL 등)와 Hadoop/BigQuery 계열은 SQL을 매개로 Python, R, Excel 등과 연결되어 분석에 활용된다.

DBMS 접속 방법

1. DBeaver

- Community Edition은 라이선스(Apache License)가 무료
- 자바/이클립스 기반, 윈도우·리눅스·MAC 구동
- JDBC 기반으로 매우 많은 DB를 지원(ORACLE, MySQL, MariaDB, PostgreSQL, Greenplum 등)
- 오픈소스라 버그픽스·신규 기능 개발이 가능
- 사용 예시(MySQL): Database Navigator에서 새 Connection 생성 → MySQL 선택 → Server Host/Port/Username/Password 입력 → Test Connection → Finish
- 자주 나는 예라: “Public Key Retrieval is not allowed” → Driver properties 탭에서 `allowPublicKeyRetrieval` 을 `true` 로 설정하면 해결

2. VS Code

- 예시: MySQL Shell for VS Code 확장(Oracle Corporation 제공)을 설치하면 VS Code 안에서 SQL 편집·실행이 가능
- `\about` , `SHOW DATABASES`; 같은 명령을 DB Notebook에서 바로 실행 가능

데이터 모델링 & ERD

데이터 모델링이란?

데이터 모델링이란 정보시스템 구축의 대상이 되는 업무 내용을 분석하여 이해하고, 약속된 표기법에 의해 표현하는 것을 의미한다. 이렇게 분석된 모델을 가지고 실제 데이터 베이스를 생성하여 개발 및 데이터 관리에 사용한다.

데이터를 추상화한 데이터 모델은 데이터베이스의 골격을 이해하고, 그 이해를 바탕으로 SQL문을 기능·성능적 측면에서 효율적으로 작성할 수 있게 해주기 때문에, 데이터 모델링은 데이터베이스 설계의 핵심 과정이다.

흐름: 현실 세계(개념1,2,3) → ① 정보 모델링 → 개념적 모델(ER 다이어그램: 개체1-관계-개체2) → ② 데이터 모델링 → 논리적 모델(관계 데이터 모델: 테이블1(속성1,속성2,속성3), 테이블2(...)) → ③ DB로 구현 → 데이터베이스(실제 테이블)

데이터 모델링 순서

1. 업무파악(요구사항 수집 및 분석)

어떤 업무를 시작하기 전에 해당 업무를 파악하는 단계. 모델링에 앞서 가장 먼저 해야 할 것은 어떤 업무를 데이터화하여 모델링할 것인지에 대한 요구사항 수집이다. 업무파악을 하기 좋은 방법은 UI를 의뢰인과 함께 확인해 나가는 것이며, 궁극적으로 만들어야 하는 것이 무엇인지 심도있게 알아보아야 한다.

2. 개념적 데이터 모델링(ERD 작성)

개념적 모델링은 Entity를 도출하고 ERD를 작성하는 단계이다. (예: 회원정보-작성-게시글 관계를 아이디, 닉네임, 비밀번호, 게시글uid, 글제목 등의 속성과 함께 다이어그램으로 표현)

3. 논리적 데이터 모델링

논리적 모델링은 ERD를 사용할 특정 DBMS의 논리적 자료구조에 맞게 사상(Mapping)하는 과정이다. 예를 들어 RDBMS를 사용한다면 ERD를 RDB로 사상하며, 테이블 설계와 정규화도 여기서 이루어진다. (예: member_tbl, board_tbl 형태로 테이블 컬럼과 타입을 확정)

4. 물리적 데이터 모델링

물리적 데이터 모델링은 최종적으로 데이터를 관리할 데이터베이스를 선택하고, 선택한 데이터베이스에 실제 테이블을 만드는 작업을 말한다. 논리적 DB 설계(엔티티→테이블, 속성→컬럼, 주식별자→기본키, 외래식별자→외래키)가 물리적 DB 설계(테이블, 컬럼, 기본키, 외래키, 뷰, 인덱스)로 이어진다.

데이터 모델링 절차 정리

① 요구사항 수집 및 분석(사용자 식별, DB 용도 식별, 사용자 요구사항 수집 및 명세) → 설계(② 개념적 모델링: 핵심 Entity 도출, ERD 작성 → DBMS 선정 → ③ 논리적 모델링: ERD-RDB 모델 사상, 상세 속성 정의, 정규화 등 → ④ 물리적 모델링: DB 개체 정의, 테이블 및 인덱스 등 설계) → 데이터베이스 구현

ERD (Entity-Relation Diagram, 개체-관계 다이어그램)

데이터베이스 설계 단계에서 맨 처음 단계인 개념적 모델링 단계에서 ERD를 작성하게 된다. ERD 표기법에는 Peter-Chen 표기법, IE 표기법 등이 있으며, 그 중 IE 표기법이 가장 많이 사용된다.

Entity(개체): 현실에 존재하는 개별적으로 식별할 수 있는 물리적 또는 추상적인 개체. 데이터베이스의 테이블이 엔티티로 표현된다고 보면 된다. 예: ‘학생’ Entity는 ‘학번’, ‘학생 이름’ 등의 Attribute를 가질 수 있고, ‘수업’ Entity는 ‘학수번호’, ‘수업 이름’ 등의 Attribute를 가질 수 있다.

ER(개체-관계) 모델: Entity 사이의 Relation(관계)을 통해 현실 세계를 표현하기 위한 설계 방식. 예: ‘학생’과 ‘수업’ Entity끼리는 ‘수강하다’라는 관계를 맺을 수 있다.

Attribute(속성): 엔티티가 갖고 있는 속성을 포함한다. 데이터베이스 테이블의 각 필드(컬럼)들이 엔티티 속성이라고 보면 된다. (예: 학생 Entity → 학번, 이름, 주소, 전공)

주 식별자(PK, Primary Key): 데이터베이스 테이블의 Primary Key를 표현. 중복이 없고 NULL 값이 없는 유일한 값에 지정하는 식별자.

외래 식별자(FK, Foreign Key): 데이터베이스 테이블의 Foreign Key를 표현. 외래 식별자를 표시할 때는 선을 이어주는데, 개체와 관계를 따져 표시한다.

Domain(도메인): 속성의 값, 타입, 제약사항 등에 대한 값의 범위를 표현하는 것. 데이터 타입을 명시할 때는 데이터베이스가 지원하는 타입에 맞게 해야 한다.

제약 조건(NOT NULL): 해당 속성에 들어갈 값에 Null을 비허용한다면 N 혹은 NN을 적는다. Null을 허용한다면 N을 적지 않는다.

Relation(관계)

- **식별관계:** 부모-자식 관계에서 자식이 부모의 주 식별자를 외래 식별자로 참조해서 **자신의 주 식별자로 설정**하는 관계
- **비식별관계:** 부모-자식 관계에서 자식이 부모의 주 식별자를 외래 식별자로 참조해서 **일반 속성으로 사용**하는 관계

Mapping Cardinality(대응수): 특정 Entity가 상대 Entity와 관계를 몇 회 맺을 수 있는지를 나타낸다.

- 학생과 학급의 소속 관계: 한 학급에는 여러 학생이 소속될 수 있지만, 한 학생은 여러 학급에 소속될 수 없다 → Cardinality는 N:1
- 학생과 짝공 관계: 서로 1회씩만 짝공 관계를 맺을 수 있다 → Cardinality는 1:1
- 학생과 동아리 소속 관계: 하나의 학생은 여러 동아리에, 하나의 동아리는 여러 학생을 소속시킬 수 있다 → Cardinality는 N:N

관계 표기 심볼

Symbol 형태	의미
One-필수	반드시 하나 있어야 함
Many-필수	반드시 하나 이상 있어야 함
One-선택	하나 있을 수도 있음(없을 수도 있음)
Many-선택	여러 개 있을 수도 있음(없을 수도 있음)

대표 관계 유형 예시(SHOP-FOOD)

- **1:1 관계:** 부모(SHOP)는 하나의 자식(FOOD)이 있다.
- **1:N 관계:** 부모(SHOP)는 하나 이상의 자식(FOOD)이 있다.
- **M:N 관계:** 하나 이상의 부모와 하나 이상의 자식이 있다.
- **1:1(o) 관계:** 부모는 하나의 자식이 있을 수도 있다(없을 수도 있다).
- **1:N(o) 관계:** 부모는 여러 개의 자식이 있을 수도 있다(없을 수도 있다).

ERD 다이어그램 툴: ERD CLOUD (웹 기반 무료 ERD 작성 툴)

DBeaver ERD 생성: 테이블 목록에서 우클릭 → “다이어그램 보기”를 선택하면 기존 DB의 테이블 관계를 자동으로 ERD로 시각화해 준다(엔티티 관계도).

데이터베이스 정규화

데이터베이스 정규화란?

데이터베이스 설계에서 데이터의 중복을 최소화하게 데이터를 구조화하는 프로세스를 정규화라고 한다. 정규화의 목표는 이상(anomaly)이 있는 관계를 재구성해서 작고 잘 조직된 관계를 생성하는 것이다.

정규화의 목적

- 불필요한 데이터를 제거하고 데이터의 중복을 최소화함으로써 저장공간이나 자원을 효율적으로 사용하기 위함
- 삽입/갱신/삭제 시 발생할 수 있는 이상 현상을 방지하기 위함

정규화 과정: 비정규 릴레이션 → (도메인이 원자값) → 1NF → (부분적 함수 종속 제거) → 2NF → (이행적 함수 종속 제거) → 3NF → (결정자이면서 후보키가 아닌 것 제거) → BCNF → (다치 종속 제거) → 4NF → (조인 종속성 이용) → 5NF

일반적으로 실무에서는 보통 **1NF ~ 3NF까지 진행**하거나 **BCNF 단계까지** 진행한다.

제1정규형(1NF)

제1 정규화란 테이블의 컬럼이 원자값(Atomic Value, 하나의 값)을 갖도록 테이블을 분해하는 것이다.

예: 고객취미들(이름, 취미들) 테이블에서 '추신수'의 취미가 “영화, 음악”처럼 한 셀에 여러 값이 들어있으면 → 고객취미(이름, 취미) 로 분해하여 (추신수, 영화), (추신수, 음악)처럼 한 행에 하나의 값만 들어가도록 만든다.

제2정규형(2NF)

제2 정규화란 제1 정규화를 진행한 테이블에 대해 **완전 함수 종속**을 만족하도록 테이블을 분해하는 것이다. 완전 함수 종속이란 기본키의 부분집합이 결정자가 되어서는 안 된다는 것을 의미한다.

예: 수강강좌(학생번호, 강좌이름, 강의실, 성적) 테이블에서 기본키는 (학생번호, 강좌이름)인 복합키이고, 이 복합키가 성적을 결정한다. 그런데 강의실 컬럼은 기본키의 부분집합인 강좌이름만으로 결정될 수 있다(강좌이름 → 강의실). 이는 부분 함수 종속이므로, 수강(학생번호, 강좌이름, 성적) 테이블과 강의실(강좌이름, 강의실) 테이블로 분해해야 한다.

제3정규형(3NF)

제3 정규화란 제2 정규화를 진행한 테이블에 대해 **이행적 종속**을 없애도록 테이블을 분해하는 것이다. 이행적 종속이란 $A \rightarrow B, B \rightarrow C$ 가 성립할 때 $A \rightarrow C$ 가 성립되는 것을 의미한다.

예: 계절학기(학생번호, 강좌이름, 수강료) 테이블에서 학생번호는 강좌이름을 결정하고, 강좌이름은 수강료를 결정한다(학생번호 → 강좌이름 → 수강료). 따라서 (학생번호, 강좌이름) 테이블과 (강좌이름, 수강료) 테이블로 분해해야 한다.

BCNF 정규화(Boyce-Codd Normal Form)

BCNF 정규화란 제3 정규화를 진행한 테이블에 대해 **모든 결정자가 후보키가 되도록** 테이블을 분해하는 것이다.

예: 특강수강(학생번호, 특강이름, 교수) 테이블에서 기본키는 (학생번호, 특강이름)이며 이 기본키가 교수를 결정한다. 그런데 교수는 특강이름도 결정한다(교수 → 특강이름). 문제는 교수가 특강이름을 결정하는 결정자이지만 후보키가 아니라는 점이다. 이를 만족시키기 위해 특강신청(학생번호, 교수) 테이블과 특강교수(특강이름, 교수) 테이블로 분해해야 한다.

Day 05 — API정의서

자료 출처: API_설계와_AI_서비스_연동.md

API 설계와 AI 서비스 연동

AI 기획자는 API를 직접 개발하는 사람이 아니라, API를 통해 어떤 데이터와 기능이 연결되어야 하는지 설명하는 사람입니다.

학습 목표

- REST API가 무엇인지 쉬운 말로 설명할 수 있다.
- API 정의서에 어떤 내용을 적어야 하는지 이해할 수 있다.
- AI 서비스에서 API가 어떤 역할을 하는지 설명할 수 있다.
- API를 기획할 때 조심해야 할 점을 알 수 있다.

오늘의 핵심 질문

오늘은 API 코드를 작성하는 시간이 아닙니다.

AI 기획자가 다음 질문에 답할 수 있도록 만드는 시간입니다.

- AI 서비스는 어떤 데이터를 어디에서 가져오는가?
- 화면과 서버는 어떻게 정보를 주고받는가?
- AI 기능을 사용하려면 어떤 요청이 필요한가?
- API가 실패하면 사용자는 무엇을 보게 되는가?
- API 정의서에는 무엇을 적어야 하는가?

API란?

API는 서로 다른 시스템이 정해진 방식으로 데이터를 주고받기 위한 약속입니다.

쉽게 말하면,

```
사용자 화면
↓ 요청
API
↓ 응답
서비스 화면에 결과 표시
```

API가 있으면 화면, 서버, 데이터, AI 모델이 서로 연결될 수 있습니다.

[원본 슬라이드 이미지 - 참고용, 본 문서에는 미포함]
w:900

API를 일상적으로 이해하기

API는 식당 주문서와 비슷합니다.

식당 주문	API 요청
메뉴판을 본다	API 문서를 본다
메뉴 이름을 말한다	요청 주소를 사용한다
옵션을 말한다	요청값을 보낸다
음식을 받는다	응답 데이터를 받는다
주문이 안 되면 이유를 듣는다	오류 메시지를 받는다

AI 서비스에서 API가 필요한 이유

AI 서비스는 혼자 동작하지 않습니다.

여러 시스템이 API로 연결됩니다.

- 화면에서 사용자의 질문을 보낸다.
- 서버가 사용자 정보를 확인한다.
- 데이터베이스에서 필요한 데이터를 가져온다.
- AI 모델에 질문을 보낸다.
- AI 답변을 화면으로 돌려준다.
- 사용 기록을 저장한다.

REST API란?

REST API는 웹에서 많이 사용하는 API 방식입니다.

REST API는 보통 다음처럼 동작합니다.

```
정해진 주소로 요청한다
↓
필요한 값을 함께 보낸다
↓
서버가 처리한다
↓
결과를 JSON 같은 형태로 응답한다
```

[원본 슬라이드 이미지 - 참고용, 본 문서에는 미포함]
w:1100

REST API의 핵심 요소

REST API를 이해할 때는 네 가지를 보면 됩니다.

요소	의미
URL	어디로 요청할 것인가
Method	무엇을 할 것인가
Request	어떤 값을 보낼 것인가
Response	어떤 결과를 받을 것인가

URL은 요청 주소다

URL은 API를 호출하는 주소입니다.

[원본 슬라이드 이미지 - 참고용, 본 문서에는 미포함]
alt text

예를 들어,

목적	URL 예시
상품 목록 보기	/products
특정 상품 보기	/products/{product_id}
사용자 정보 보기	/users/{user_id}
추천 결과 받기	/recommendations
챗봇 답변 받기	/chat

URL은 기능의 위치를 알려 줍니다.

Method는 요청 행동이다

Method는 API로 무엇을 할지 나타냅니다.

Method	의미	예시
GET	조회하기	상품 목록 보기
POST	새로 만들기 또는 요청하기	AI 추천 요청
PUT/PATCH	수정하기	사용자 정보 수정
DELETE	삭제하기	저장된 항목 삭제

기획자는 Method 이름을 외우기보다, 조회인지 요청인지 수정인지 구분하면 됩니다.

[원본 슬라이드 이미지 - 참고용, 본 문서에는 미포함]
w:1200

Request는 보내는 값이다

Request는 API를 호출할 때 함께 보내는 값입니다.

예를 들어 추천 API라면 다음 값이 필요할 수 있습니다.

요청값	의미
user_id	어떤 사용자에게 추천할지
keyword	어떤 관심사를 기준으로 볼지
limit	몇 개를 추천받을지
exclude	제외할 조건이 있는지

Response는 받는 값이다

Response는 API가 처리 후 돌려주는 결과입니다.

예를 들어 추천 API의 응답은 다음 정보를 포함할 수 있습니다.

응답값	의미
item_id	추천 대상 ID
item_name	추천 대상 이름
reason	추천 이유
score	추천 점수
source	추천 근거 데이터

AI 서비스에서는 응답값이 화면에 어떻게 보일지도 함께 생각해야 합니다.

JSON이란?

JSON은 API에서 데이터를 주고받을 때 자주 쓰는 형식입니다.

사람이 읽기 쉬운 이름과 값의 묶음이라고 이해하면 됩니다.

```
{
  "question": "배송은 언제 오나요?",
  "answer": "주문 내역에서 배송 상태를 확인할 수 있습니다."
}
```

기획자는 JSON을 깊게 작성할 필요는 없지만, 어떤 값이 오가는지는 읽을 수 있어야 합니다.

API 정의서란?

API 정의서는 API를 어떻게 사용할지 정리한 문서입니다.

개발자만 보는 문서가 아니라, 기획자와 디자이너도 함께 이해해야 하는 약속입니다.

API 정의서에는 보통 다음 내용이 들어갑니다.

- API 이름
- API 목적
- 요청 주소
- 요청 방식
- 요청값
- 응답값
- 오류 상황
- 인증 여부

API 정의서가 필요한 이유

API 정의서가 없으면 사람마다 다르게 이해할 수 있습니다.

예를 들어,

- 화면에서는 10개 결과를 기대하는데 API는 5개만 준다.
- 기획자는 추천 이유가 필요하지만 API 응답에는 없다.
- 사용자가 입력하지 않은 값이 API에서는 필수값이다.
- 실패했을 때 어떤 메시지를 보여줄지 정해져 있지 않다.

API 정의서는 이런 차이를 줄입니다.

API 정의서 기본 구조

항목	설명
API 이름	어떤 API인지
목적	왜 필요한 API인지
URL	어디로 요청하는지
Method	조회, 요청, 수정, 삭제 중 무엇인지
요청값	API를 부를 때 보내는 값
응답값	API가 돌려주는 값
오류	실패할 수 있는 상황
비고	주의할 점

요청값 정의하기

요청값은 사용자가 보내거나 화면에서 전달하는 값입니다.

요청값	설명	필수 여부	예시
user_id	사용자 식별값	필수	U001
question	사용자의 질문	필수	환불은 어떻게 하나요?
category	검색 범위	선택	배송
page	페이지 번호	선택	1

필수값이 많을수록 사용자는 더 많은 정보를 입력해야 합니다.

응답값 정의하기

응답값은 화면이나 AI 기능이 사용할 결과입니다.

응답값	설명	예시
-----	----	----

answer	AI 답변	환불은 주문 내역에서 신청할 수 있습니다.
source	답변 근거	환불 정책 문서
confidence	답변 신뢰도	높음
created_at	응답 생성 시각	2026-07-11

AI 서비스에서는 답변 내용뿐 아니라 근거와 상태도 중요합니다.

오류 상황도 정의해야 한다

API는 항상 성공하지 않습니다.

실패 상황도 미리 정의해야 합니다.

오류 상황	사용자에게 필요한 안내
필수값이 없음	필요한 정보를 다시 입력해 주세요
데이터가 없음	조건에 맞는 결과가 없습니다
권한이 없음	로그인 또는 권한 확인이 필요합니다
외부 API 실패	잠시 후 다시 시도해 주세요
AI 답변 실패	답변을 생성하지 못했습니다

인증이란?

인증은 API를 아무나 쓰지 못하게 확인하는 과정입니다.

예를 들어,

- 로그인한 사용자만 호출할 수 있다.
- 관리자만 수정할 수 있다.
- API Key가 있어야 외부 데이터를 가져올 수 있다.
- 사용자의 개인정보는 본인만 볼 수 있다.

기획자는 어떤 API에 인증이 필요한지 구분해야 합니다.

AI 서비스에서 API가 쓰이는 곳

AI 서비스에서는 다양한 API가 필요할 수 있습니다.

API	역할
사용자 API	사용자 정보 조회
데이터 API	상품, 문서, 기록 조회
추천 API	맞춤 추천 결과 생성
챗봇 API	사용자 질문에 대한 답변 생성
검색 API	관련 문서나 콘텐츠 검색
로그 API	사용 기록 저장

AI 모델 API

AI 모델을 사용할 때도 API를 호출하는 경우가 많습니다.

흐름은 단순하게 보면 다음과 같습니다.



↓
사용자 화면

기획자는 어떤 질문을 보내고, 어떤 답변을 받아야 하는지 정의해야 합니다.

AI 답변 API에서 중요한 값

AI 답변 API를 설계할 때는 답변 텍스트만 생각하면 부족합니다.

값	이유
사용자 질문	AI가 답할 내용
대화 기록	앞뒤 맥락 유지
참고 문서	근거 있는 답변 생성
답변 내용	사용자에게 보여줄 결과
출처	답변 근거 표시
실패 메시지	답변 불가 상황 안내

RAG와 API

RAG는 AI가 답변하기 전에 관련 문서를 찾아 참고하는 방식입니다.

이때도 API가 사용될 수 있습니다.

```
사용자 질문
↓
검색 API로 관련 문서 찾기
↓
AI 모델 API에 질문과 문서 전달
↓
근거 있는 답변 생성
```

AI 기획자는 어떤 문서를 검색하고, 답변에 출처를 보여줄지 정해야 합니다.

API가 느리면 AI 서비스도 느려진다

API 속도는 사용자 경험에 영향을 줍니다.

특히 AI 서비스는 응답 시간이 길어질 수 있습니다.

기획자는 다음을 고려해야 합니다.

- 사용자가 얼마나 기다릴 수 있는가?
- 로딩 화면이 필요한가?
- 답변 생성 중이라는 안내가 필요한가?
- 너무 오래 걸리면 중단할 것인가?
- 실패 시 다시 시도할 수 있는가?

API 호출 제한

외부 API는 호출 횟수 제한이 있을 수 있습니다.

예를 들어,

- 하루 1,000번까지만 호출 가능
- 1분에 60번까지만 호출 가능
- 사용량이 많아지면 비용 증가
- 무료 한도를 넘으면 응답 중단

AI 서비스는 사용자가 늘수록 API 사용량도 늘어날 수 있습니다.

API 정의서 작성 시 기획자 체크리스트

API를 정의할 때 다음을 확인합니다.

- 이 API는 어떤 기능 때문에 필요한가?
- 요청값은 무엇인가?
- 필수값과 선택값은 무엇인가?
- 응답값은 화면에 충분한가?
- 실패하면 어떤 메시지를 보여줄 것인가?
- 인증이 필요한가?
- 호출 제한이나 비용이 있는가?
- AI 답변 근거를 표시할 수 있는가?

좋지 않은 API 정의서

다음과 같은 API 정의서는 협업을 어렵게 만듭니다.

- API 목적이 없다.
- 요청값과 응답값이 모호하다.
- 예시가 없다.
- 필수값이 표시되어 있지 않다.
- 오류 상황이 없다.
- 화면에서 필요한 값이 응답에 없다.
- AI 답변 실패 상황이 고려되어 있지 않다.

좋은 API 정의서

좋은 API 정의서는 읽는 사람이 같은 기능을 떠올릴 수 있습니다.

- API 이름이 명확하다.
- 어떤 화면이나 기능에서 쓰는지 알 수 있다.
- 요청값과 응답값이 구체적이다.
- 예시가 있다.
- 오류 상황과 사용자 안내가 있다.
- 인증과 권한이 표시되어 있다.
- AI 답변의 근거와 실패 상황을 고려한다.

너무 어렵게 생각하지 않기

AI 기획자가 API 개발자가 될 필요는 없습니다.

다만 다음 정도는 설명할 수 있어야 합니다.

- 어떤 기능에 API가 필요한지
- 화면에서 어떤 값을 보내야 하는지
- API가 어떤 값을 돌려줘야 하는지
- 실패하면 사용자에게 어떻게 안내할지
- AI 기능이 어떤 데이터와 연결되는지

핵심 정리

- API는 시스템끼리 데이터를 주고받기 위한 약속이다.
- REST API는 URL, Method, Request, Response를 중심으로 이해하면 된다.
- API 정의서는 API의 목적, 요청값, 응답값, 오류 상황을 정리한 문서다.
- AI 서비스에서는 사용자 정보, 데이터 조회, 검색, 추천, 챗봇, AI 모델 호출에 API가 쓰인다.
- AI 기획자는 API를 개발하지 않아도, API가 어떤 기능과 데이터를 연결하는지 설명할 수 있어야 한다.

오늘 기억할 문장

API 정의서는 개발자만을 위한 문서가 아닙니다.

AI 기획자가 서비스 기능과 데이터, AI 답변 흐름을 연결하기 위해 필요한 약속입니다.

3단계. 서비스설계 & 프로토타입 (day01 ~ day05)

Day 01 — MVP정의

자료 출처: MVP와_서비스_기능_정의.md

MVP와 서비스 기능 정의

오늘의 핵심 질문

- 우리가 만들 서비스의 첫 버전은 어디까지여야 할까?
- 사용자가 정말 필요로 하는 기능은 무엇일까?
- 모든 기능을 한 번에 만들 수 없다면 무엇부터 선택해야 할까?
- AI 기능은 언제 넣어야 하고, 언제 미뤄야 할까?

AI 기획자에게 기능 정의가 중요한 이유

AI 서비스는 아이디어가 커지기 쉽습니다.

- 챗봇도 넣고 싶다
- 추천도 넣고 싶다
- 자동 분석도 넣고 싶다
- 관리자 화면도 필요해 보인다
- 데이터 대시보드도 있으면 좋겠다

하지만 첫 버전의 목표는 “많이 만드는 것”이 아니라
사용자의 핵심 문제가 실제로 풀리는지 확인하는 것입니다.

기능 정의란 무엇인가

기능 정의는 서비스를 구성하는 일을 사용자 관점에서 정리하는 과정입니다.

기능 정의는 단순한 기능 목록이 아닙니다.

구분	질문
사용자	누가 사용하는가?
상황	언제, 어떤 문제 때문에 사용하는가?
행동	사용자가 무엇을 할 수 있어야 하는가?
가치	그 행동 후 무엇이 좋아지는가?
범위	첫 버전에 어디까지 포함할 것인가?

기능을 먼저 많이 적으면 생기는 문제

“있으면 좋은 기능”이 많아질수록 서비스는 흐려집니다.

- 핵심 사용자가 누구인지 모호해진다
- 첫 출시 범위가 커진다
- 개발 기간과 비용이 늘어난다
- AI 기능의 품질 검증이 어려워진다
- 사용자가 실제로 원하는 가치보다 화면과 기능 수에 집중하게 된다

AI 기획자는 기능을 늘리는 사람이 아니라
사용자 가치에 맞게 기능을 줄이고 정렬하는 사람입니다.

좋은 기능 정의의 기준

좋은 기능 정의는 다음 네 가지를 설명할 수 있어야 합니다.

1. 이 기능은 누구를 위한 것인가?
2. 사용자는 이 기능으로 어떤 문제를 해결하는가?
3. 이 기능이 없으면 첫 버전의 가치가 무너지는가?
4. 이 기능은 지금 만들 만큼 중요한가?

기능의 이름보다 중요한 것은

그 기능이 사용자에게 만들어 주는 변화입니다.

User Story란 무엇인가

User Story는 기능을 사용자 관점의 짧은 문장으로 표현하는 방법입니다.

기본 형식은 다음과 같습니다.

나는 [사용자]로서
[원하는 행동]을 하고 싶다.
그래서 [얻고 싶은 가치]를 얻을 수 있다.

User Story는 기능명을 정리하는 도구가 아니라

기능이 필요한 이유를 분명하게 만드는 도구입니다.

[원본 슬라이드 이미지 - 참고용, 본 문서에는 미포함]

w:1100

User Story의 세 부분

구성	의미	예시
사용자	누구의 문제인가	바쁜 직장인
원하는 행동	무엇을 하고 싶은가	오늘 먹을 건강한 식단을 빠르게 추천받고 싶다
얻는 가치	왜 필요한가	메뉴 고민 시간을 줄이고 식습관을 관리할 수 있다

세 부분 중 하나라도 비어 있으면 기능의 목적이 약해집니다.

User Story 예시

나쁜 예시:

사용자는 AI 추천 기능을 사용할 수 있다.

좋은 예시:

나는 식단 관리가 필요한 직장인으로서
내 알레르기과 선호 음식을 반영한 점심 메뉴를 추천받고 싶다.
그래서 안전하고 지속 가능한 식사를 선택할 수 있다.

차이는 기능명이 아니라

사용자 상황과 가치가 보이는가에 있습니다.

User Story를 쓸 때 피해야 할 표현

다음 표현은 기능을 너무 공급자 관점으로 만들 수 있습니다.

- AI를 적용한다
- 대시보드를 만든다

- 데이터를 시각화한다
- 추천 알고리즘을 구현한다
- 관리자 기능을 제공한다

이 표현이 틀린 것은 아닙니다.
다만 User Story에서는 먼저 사용자 관점으로 바꿔야 합니다.

[원본 슬라이드 이미지 - 참고용, 본 문서에는 미포함]
w:1100

공급자 관점을 사용자 관점으로 바꾸기

공급자 관점	사용자 관점
AI 추천 기능 제공	사용자가 선택하기 어려운 상황에서 적절한 후보를 빠르게 고른다
대시보드 제공	사용자가 현재 상태를 한눈에 이해하고 다음 결정을 내린다
알림 기능 제공	사용자가 중요한 시점을 놓치지 않는다
검색 기능 제공	사용자가 원하는 정보를 빠르게 찾는다
데이터 분석 제공	사용자가 판단에 필요한 근거를 확인한다

기능은 수단이고, 사용자 가치는 목적입니다.

User Story가 기능 정의에 주는 도움

User Story를 쓰면 다음 판단이 쉬워집니다.

- 이 기능이 어떤 사용자를 위한 것인지 구분할 수 있다
- 기능의 목적이 흐려지는 것을 막을 수 있다
- 팀 안에서 같은 기능을 다르게 이해하는 문제를 줄일 수 있다
- MVP에 포함할 기능과 나중에 만들 기능을 나눌 수 있다
- AI 기능이 “멋져 보여서”가 아니라 “문제를 풀어서” 필요한지 확인할 수 있다

MVP란 무엇인가

MVP는 Minimum Viable Product의 약자입니다.

한국어로는 보통 “최소 기능 제품”이라고 부릅니다.

하지만 더 정확히는 다음 뜻에 가깝습니다.

사용자의 핵심 문제를 해결하는 데 필요한
가장 작은 검증 가능한 서비스 버전

MVP는 작게 만드는 것이 목적이 아니라
중요한 가설을 빠르게 확인하는 것이 목적입니다.

[원본 슬라이드 이미지 - 참고용, 본 문서에는 미포함]
w:800

MVP에서 Minimum의 의미

Minimum은 아무렇게나 적게 만든다는 뜻이 아닙니다.

Minimum은 다음을 뜻합니다.

- 핵심 사용자에게 꼭 필요한 것만 남긴다
- 첫 검증에 필요 없는 기능은 미룬다
- 사용자가 가치를 느끼는 흐름은 끊기지 않게 한다
- 보기 좋은 부가 기능보다 문제 해결을 우선한다

즉, MVP는 “작지만 쓸 수 있는 서비스”여야 합니다.

MVP에서 Viable의 의미

Viable은 실제로 가치가 있어야 한다는 뜻입니다.

너무 적게 만들면 MVP가 아니라 불완전한 화면이 됩니다.

Viable한 MVP는 다음 조건을 갖습니다.

- 사용자가 자신의 문제를 해결해 볼 수 있다
- 서비스의 핵심 흐름이 처음부터 끝까지 이어진다
- 결과가 사용자의 판단이나 행동에 도움을 준다
- 피드백을 받을 수 있을 만큼 구체적이다

MVP는 “반쪽짜리 서비스”가 아니라

핵심 가치만 남긴 첫 서비스입니다.

MVP가 아닌 것

다음은 MVP와 혼동하기 쉽습니다.

오해	왜 다른가
기능을 대충 줄인 버전	사용자 가치가 끊기면 검증이 어렵다
디자인이 없는 화면	화면 완성도가 아니라 문제 해결 흐름이 중요하다
기술 데모	기술이 작동해도 사용자가 가치를 느끼는지는 별도 문제다
모든 기능의 축소판	모든 것을 조금씩 넣으면 핵심이 흐려진다
출시 전 임시 결과물	MVP도 실제 사용자 반응을 확인하기 위한 서비스 버전이다

AI 서비스의 MVP는 더 조심해야 한다

AI 기능은 매력적이지만 불확실성이 큼니다.

- 답변 품질이 일정하지 않을 수 있다
- 데이터가 충분하지 않을 수 있다
- 사용자가 AI 결과를 신뢰하지 않을 수 있다
- 설명이 부족하면 오히려 불안해질 수 있다
- 자동화 범위가 커질수록 오류 책임이 커진다

그래서 AI MVP에서는

AI가 꼭 필요한 순간과 사람이 판단해야 할 순간을 나눠야 합니다.

AI 기능을 MVP에 넣어도 되는 경우

다음 질문에 “그렇다”라고 답할 수 있으면 MVP에 포함할 가능성이 높습니다.

- 이 AI 기능이 없으면 핵심 사용자 문제가 해결되지 않는가?
- AI가 사용자의 시간을 실제로 줄여 주는가?
- 결과가 틀렸을 때 사용자가 확인하거나 수정할 수 있는가?
- AI 결과의 품질을 비교할 기준이 있는가?
- 단순 규칙이나 검색보다 AI가 더 나은 이유가 분명한가?

AI는 차별화를 위한 장식이 아니라

사용자 문제를 푸는 수단이어야 합니다.

기능 후보를 정리하는 방식

기능을 바로 우선순위로 나누기 전에 먼저 묶어 봅니다.

기능 묶음	질문
사용 시작	사용자가 어떻게 들어오는가?
정보 입력	사용자가 무엇을 알려 주어야 하는가?

핵심 결과	서비스가 어떤 결과를 제공하는가?
확인과 수정	사용자가 결과를 어떻게 검토하는가?
저장과 공유	결과를 다시 보거나 전달해야 하는가?
운영 관리	서비스 운영자가 무엇을 확인해야 하는가?

기능 후보는 화면 단위가 아니라 **사용자 흐름 단위**로 정리하는 것이 좋습니다.

기능 우선순위가 필요한 이유

모든 기능을 동시에 만들 수는 없습니다.

우선순위는 다음을 정하기 위해 필요합니다.

- 첫 버전에 반드시 들어갈 기능
- 있으면 좋지만 나중에 넣어도 되는 기능
- 사용자 검증 후 결정할 기능
- 지금 만들면 오히려 복잡해지는 기능

우선순위는 단순한 중요도 순서가 아닙니다.

첫 검증에 필요한 범위를 정하는 의사결정입니다.

MoSCoW란 무엇인가

MoSCoW는 기능 우선순위를 네 단계로 나누는 방법입니다.

구분	의미
Must have	반드시 있어야 하는 기능
Should have	중요하지만 첫 버전에서 조정 가능한 기능
Could have	있으면 좋지만 없어도 핵심 가치는 유지되는 기능
Won't have	이번 범위에서는 제외할 기능

MoSCoW의 핵심은 **모든 기능을 중요하다고 말하지 않는 것**입니다.

[원본 슬라이드 이미지 - 참고용, 본 문서에는 미포함]

w:1100

Must have

Must have는 이 기능이 없으면 서비스의 핵심 가치가 성립하지 않는 기능입니다.

판단 질문:

- 이 기능이 없으면 사용자가 핵심 문제를 해결할 수 없는가?
- 이 기능이 없으면 User Story가 완성되지 않는가?
- 이 기능이 없으면 MVP 검증이 의미 없어지는가?

Must have는 적을수록 좋습니다.

Must가 많아지면 MVP가 아니라 전체 서비스에 가까워집니다.

Should have

Should have는 중요하지만 첫 검증에서 반드시 필요한 것은 아닌 기능입니다.

예를 들어:

- 결과 화면의 세부 필터

- 더 편한 입력 방식
- 추가 설명 문구
- 보조 알림
- 관리자용 상세 통계

Should have는 사용자 경험을 좋아지게 만들지만
없어도 핵심 흐름이 작동해야 합니다.

Could have

Could have는 있으면 만족도가 올라가지만 우선순위가 낮은 기능입니다.

예를 들어:

- 테마 색상 선택
- 프로필 꾸미기
- 고급 공유 옵션
- 추가 추천 카드
- 보조 콘텐츠 영역

Could have는 나쁘다는 뜻이 아닙니다.
다만 첫 버전에서 핵심 검증보다 앞서면 안 됩니다.

Won't have

Won't have는 이번 범위에서는 의도적으로 제외하는 기능입니다.

제외는 포기가 아닙니다.

- 지금 만들기에 검증 근거가 부족하다
- 핵심 사용자 문제와 거리가 있다
- 운영 부담이 크다
- 품질을 보장하기 어렵다
- 다음 버전에서 판단하는 것이 낫다

좋은 기획자는 무엇을 넣을지뿐 아니라
무엇을 지금 넣지 않을지도 설명할 수 있어야 합니다.

MoSCoW 예시

건강 식단 추천 서비스를 예로 들어 봅니다.

구분	기능 예시	이유
Must	선호 음식과 알레르기 입력	안전한 추천의 기본 조건
Must	오늘의 식단 추천 결과	서비스의 핵심 가치
Should	추천 이유 설명	사용자의 신뢰 형성에 도움
Could	식단 이미지 꾸미기	만족도는 높일 수 있지만 핵심은 아님
Won't	자동 장보기 주문	운영 난이도가 높고 첫 검증 범위를 넘음

사용자 가치 중심 기능 선정

기능을 선택할 때는 “만들 수 있는가”보다 먼저 "사용자에게 가치가 있는가"를 봐야 합니다.

사용자 가치 중심 기능 선정은 다음 순서로 생각합니다.

1. 사용자의 문제를 확인한다
2. 문제를 해결하는 핵심 행동을 찾는다
3. 그 행동을 가능하게 하는 기능만 남긴다
4. 첫 검증에 필요 없는 기능을 미룬다

5. 사용자가 얻는 결과를 명확히 설명한다

사용자 가치의 네 가지 기준

기준	질문
문제 강도	사용자가 실제로 불편해하는 문제인가?
사용 빈도	반복적으로 필요해지는가?
대체 난이도	기존 방식으로 해결하기 번거로운가?
결과 명확성	기능 사용 후 좋아진 점이 분명한가?

기능의 가치는 기능 자체가 아니라
사용자의 전후 변화로 판단합니다.

기능 선정에서 자주 생기는 착각

기능이 많으면 좋은 서비스처럼 보일 수 있습니다.

하지만 사용자는 기능 수보다 다음을 더 중요하게 느낍니다.

- 내가 원하는 결과가 빨리 나오는가?
- 입력해야 할 것이 너무 많지 않은가?
- 결과를 믿을 수 있는가?
- 다음 행동을 바로 결정할 수 있는가?
- 불필요한 선택지가 나를 방해하지 않는가?

기능이 많아도 사용자 가치가 흐리면 좋은 서비스가 되기 어렵습니다.

기능을 줄이는 것은 품질을 낮추는 일이 아니다

기능을 줄이는 것은 기획의 후퇴가 아닙니다.

오히려 다음을 더 분명하게 만듭니다.

- 누구를 위한 서비스인지
- 어떤 문제를 먼저 해결하는지
- 어떤 데이터가 필요한지
- 어떤 AI 기능이 정말 필요한지
- 어떤 결과로 성공을 판단할지

기능을 줄일수록 서비스의 핵심 약속은 더 선명해집니다.

AI 기획자의 기능 선정 질문

기능 후보를 볼 때 다음 질문을 던져 봅니다.

- 이 기능은 사용자의 어떤 불편을 줄이는가?
- 이 기능은 AI가 해야 하는 일인가, 규칙으로도 가능한가?
- 사용자는 이 기능의 결과를 이해하고 신뢰할 수 있는가?
- 결과가 틀렸을 때 회복 방법이 있는가?
- 첫 버전에서 이 기능을 빼면 어떤 문제가 생기는가?
- 이 기능보다 먼저 확인해야 할 더 중요한 가설은 없는가?

User Story와 MVP를 연결하기

MVP 범위는 User Story에서 출발해야 합니다.

User Story 요소	MVP 판단
사용자	첫 버전의 핵심 사용자를 정한다
원하는 행동	반드시 가능한 핵심 흐름을 정한다
얻는 가치	성공 여부를 판단할 기준을 정한다

User Story가 흐리면 MVP도 커집니다.
User Story가 선명하면 기능 우선순위도 쉬워집니다.

MVP 기능 정의의 흐름

AI 기획자는 다음 흐름으로 첫 버전을 정리할 수 있습니다.

1. 핵심 사용자를 정한다
2. 사용자의 문제 상황을 문장으로 쓴다
3. User Story로 기능의 이유를 설명한다
4. 기능 후보를 사용자 흐름에 맞게 묶는다
5. MoSCoW로 우선순위를 나눈다
6. Must have만으로도 가치가 전달되는지 확인한다
7. AI 기능의 필요성과 위험을 따로 점검한다

좋은 MVP 설명의 예

약한 설명:

AI 기반 건강 관리 앱을 만든다.

좋은 설명:

바쁜 직장인이 자신의 알레르기과 선호 음식을 입력하면
오늘 먹을 수 있는 점심 메뉴를 추천받고,
추천 이유를 확인해 식사 선택 시간을 줄이는 첫 버전을 만든다.

좋은 설명에는 사용자, 행동, 가치, 첫 범위가 함께 들어 있습니다.

기능 우선순위 회의에서 필요한 태도

우선순위는 누가 더 강하게 주장하는지로 정하면 안 됩니다.

다음 기준으로 대화해야 합니다.

- 사용자 문제와 얼마나 가까운가
- 첫 검증에 반드시 필요한가
- 데이터와 품질을 감당할 수 있는가
- 기능이 늘어나면서 사용 흐름이 복잡해지지 않는가
- 나중에 넣어도 가치가 유지되는가

기획자는 의견을 모으는 사람이 아니라

판단 기준을 세우는 사람입니다.

오늘의 정리

- User Story는 기능을 사용자, 행동, 가치로 설명하는 문장이다
- MVP는 핵심 문제를 검증하는 가장 작은 서비스 버전이다
- MoSCoW는 기능을 Must, Should, Could, Won't로 나누는 우선순위 방법이다
- 사용자 가치 중심 기능 선정은 기능 수보다 문제 해결 효과를 우선한다
- AI 기획자는 AI 기능의 매력보다 사용자 문제, 품질, 신뢰, 검증 가능성을 먼저 본다

마지막으로 기억할 문장

첫 버전의 목표는 완벽한 서비스를 만드는 것이 아닙니다.

가장 중요한 사용자가

가장 중요한 문제를

가장 짧은 흐름으로 해결할 수 있는지 확인하는 것입니다.

Day 02 — 정보구조(IA)

자료 출처: 정보_구조_IA_및_사용자_흐름_User_Flow_.md

정보 구조(IA) 및 사용자 흐름(User Flow)

AI 서비스의 화면과 흐름을 사용자 관점으로 설계하는 방법

오늘의 핵심 질문

- 사용자는 서비스 안에서 무엇을 먼저 봐야 할까?
- 어떤 정보와 기능을 어디에 배치해야 할까?
- 사용자가 목표를 달성하기까지 어떤 흐름을 지나야 할까?
- AI 결과가 나오는 서비스에서는 어떤 확인과 예외 흐름이 필요할까?

기능이 있어도 사용하기 어려운 서비스

좋은 기능이 있어도 사용자가 찾지 못하면 가치가 전달되지 않습니다.

- 필요한 메뉴가 어디 있는지 모른다
- 입력해야 할 정보가 너무 많다
- 다음에 무엇을 해야 하는지 헷갈린다
- AI 결과를 어떻게 해석해야 할지 모르겠다
- 저장, 수정, 다시 시도 흐름이 보이지 않는다

AI 기획자는 기능을 정의한 뒤

사용자가 그 기능을 자연스럽게 사용할 수 있는 구조를 설계해야 합니다.

정보구조 (I.A: Information Architecture)란 무엇인가

IA는 서비스 안의 정보와 기능을 사용자가 이해하기 쉬운 방식으로 정리하는 설계입니다.

쉽게 말하면 다음을 정하는 일입니다.

- 어떤 정보가 있는가
- 어떤 기능이 있는가
- 무엇을 먼저 보여줄 것인가
- 어떤 메뉴나 화면에 묶을 것인가
- 사용자가 원하는 것을 어떻게 찾게 할 것인가

[원본 슬라이드 이미지 - 참고용, 본 문서에는 미포함]

alt text

IA는 화면 디자인이 아니다

IA는 화면을 예쁘게 꾸미는 일이 아닙니다.

IA는 화면이 만들어지기 전에

정보와 기능의 위치, 관계, 우선순위를 정하는 일입니다.

구분	관심사
IA	정보와 기능을 어떻게 묶고 배치할 것인가
화면 디자인	각 화면을 어떻게 보이게 만들 것인가
User Flow	사용자가 어떤 순서로 이동할 것인가
콘텐츠	사용자가 읽고 이해할 문구와 데이터

IA가 흔들리면 디자인이 좋아도 사용자는 길을 잃습니다.

AI 서비스에서 IA가 더 중요한 이유

AI 서비스는 일반 서비스보다 사용자가 확인해야 할 정보가 많습니다.

- 사용자가 입력한 조건
- AI가 만든 결과
- 결과가 나온 이유
- 결과를 수정하거나 다시 요청하는 방법
- AI가 할 수 없는 일
- 사용자가 직접 판단해야 하는 부분

AI 결과만 보여주면 충분하지 않습니다.

사용자가 결과를 이해하고 다음 행동을 할 수 있어야 합니다.

IA 설계의 기본 요소

IA를 설계할 때는 다음 요소를 함께 봅니다.

요소	설명
정보	사용자가 알아야 하는 내용
기능	사용자가 할 수 있는 행동
그룹	비슷한 정보와 기능의 묶음
우선순위	먼저 보여줄 것과 나중에 보여줄 것
이름	사용자가 이해할 수 있는 메뉴와 버튼 이름
이동	사용자가 다른 화면으로 가는 방법

좋은 IA는 사용자의 머릿속 분류와 서비스의 구조가 비슷합니다.

IA 설계의 출발점

IA는 회사가 가진 기능 목록이 아니라 사용자 목표에서 시작합니다.

질문은 이렇게 바뀌어야 합니다.

공급자 관점	사용자 관점
추천 기능을 어디에 넣을까?	사용자는 언제 추천이 필요할까?
분석 결과를 어떤 메뉴에 넣을까?	사용자는 어떤 판단을 하려고 결과를 볼까?
설정 화면을 어떻게 만들까?	사용자는 무엇을 바꾸고 싶어 할까?
히스토리를 보여줄까?	사용자는 이전 결과를 왜 다시 보려 할까?

IA의 기준은 내부 조직도가 아니라 사용자 목표입니다.

IA 설계 순서

AI 기획자는 IA를 다음 순서로 정리할 수 있습니다.

1. 핵심 사용자의 목표를 확인한다
2. 사용자가 봐야 할 정보와 사용할 기능을 모두 적는다
3. 비슷한 정보와 기능을 묶는다
4. 사용자가 자주 쓰는 것과 중요한 것을 앞에 둔다
5. 메뉴와 화면 이름을 사용자 언어로 바꾼다
6. 처음 진입부터 결과 확인까지의 큰 구조를 점검한다
7. 예외 상황과 되돌아가는 길을 확인한다

좋은 IA의 기준

좋은 IA는 사용자가 많이 생각하지 않아도 이해됩니다.

- 중요한 것이 먼저 보인다
- 메뉴 이름이 쉽고 예측 가능하다
- 비슷한 정보가 한곳에 모여 있다
- 다음 행동으로 가는 길이 보인다
- 처음 사용하는 사람도 전체 구조를 짐작할 수 있다
- 나중에 기능이 늘어나도 구조가 쉽게 무너지지 않는다

IA의 핵심은 “정리”가 아니라 “이해 가능성”입니다.

IA에서 피해야 할 구조

다음 구조는 사용자를 헷갈리게 만들 수 있습니다.

- 내부 팀 기준으로 메뉴를 나눈 구조
- 비슷한 기능이 여러 곳에 흩어진 구조
- 메뉴 이름이 너무 기술적인 구조
- 사용 빈도가 낮은 기능이 앞에 있는 구조
- AI 결과와 사용자 입력 조건이 분리되어 맥락을 잃는 구조
- 오류나 실패 상황에서 돌아갈 길이 없는 구조

사용자는 서비스 조직도를 배우고 싶어 하지 않습니다.
사용자는 자기 문제를 해결하고 싶어 합니다.

User Journey란 무엇인가

User Journey는 사용자가 어떤 상황에서 서비스를 만나고, 사용하고, 결과를 받아들이는 전체 경험의 흐름입니다.

User Journey는 화면 이동만 보지 않습니다.

- 사용 전 기대
- 사용 중 행동
- 사용 중 감정
- 막히는 지점
- 결과를 보고 느끼는 신뢰
- 사용 후 다음 행동

즉, User Journey는 사용자의 경험을 시간 순서로 보는 방법입니다.

[원본 슬라이드 이미지 - 참고용, 본 문서에는 미포함]
w:1000

User Journey가 필요한 이유

서비스는 화면 안에서만 일어나지 않습니다.

사용자는 서비스를 사용하기 전부터 이미 문제를 느끼고 있습니다.

- 메뉴를 고르기 어렵다
- 정보를 비교하기 귀찮다
- 무엇이 맞는 선택인지 확신이 없다
- 시간을 줄이고 싶다
- 실수하고 싶지 않다

User Journey는 사용자의 문제, 감정, 기대를 함께 보게 해 줍니다.

User Journey의 기본 구성

User Journey는 보통 다음 항목으로 정리합니다.

항목	질문
단계	사용자는 어떤 순서를 지나가는가?
행동	단계마다 무엇을 하는가?
생각	사용자는 무엇을 궁금해하는가?

감정	편한가, 불안한가, 답답한가?
Pain Point	어디서 막히는가?
기회	서비스가 어디서 도움을 줄 수 있는가?

기획자는 기능뿐 아니라 사용자의 심리 흐름도 읽어야 합니다.

Journey에서 봐야 할 감정 변화

기능은 작동해도 감정이 나쁘면 사용자는 떠날 수 있습니다.

AI 서비스에서는 특히 다음 감정이 중요합니다.

- 기대: AI가 내 문제를 잘 풀어 줄 것 같다
- 불안: 내 정보를 입력해도 괜찮을까?
- 의심: 이 추천을 믿어도 될까?
- 답답함: 왜 이런 결과가 나왔는지 모르겠다
- 안심: 이유와 수정 방법이 보여서 믿을 수 있다

User Journey는 이런 감정의 변화를 설계에 반영하게 해 줍니다.

User Journey와 IA의 관계

User Journey는 사용자의 경험 흐름을 보여 줍니다.

IA는 그 경험을 가능하게 하는 정보 구조를 보여 줍니다.

User Journey에서 보이는 것	IA에서 반영할 것
사용자가 처음에 불안해한다	개인정보 안내와 입력 이유를 앞에 둔다
결과를 믿기 어려워한다	추천 이유와 기준을 결과 화면에 둔다
다시 수정하고 싶어 한다	조건 수정과 재추천 경로를 가깝게 둔다
이전 결과를 다시 보고 싶어 한다	저장 목록과 히스토리 구조를 만든다

Journey는 사용자의 마음을 보여 주고, IA는 그것을 구조로 옮깁니다.

User Flow란 무엇인가

User Flow는 사용자가 특정 목표를 달성하기 위해 서비스 안에서 이동하는 경로입니다.

예를 들어:

사용자가 식단 추천을 받기 위해
홈 화면에서 시작해 조건 입력, 결과 확인, 저장까지 이동하는 흐름

User Flow는 사용자의 행동 순서와 화면 전환을 구체적으로 보여 줍니다.

[원본 슬라이드 이미지 - 참고용, 본 문서에는 미포함]
alt text

User Flow는 왜 필요한가

User Flow를 작성하면 다음을 확인할 수 있습니다.

- 사용자가 목표까지 너무 오래 걸리지 않는가
- 불필요한 단계가 끼어 있지 않은가
- 다음 버튼이나 선택지가 명확한가
- 실패하거나 취소했을 때 돌아갈 길이 있는가
- AI 결과를 다시 요청하거나 수정하는 길이 있는가

흐름을 그려 보면 기능 목록에서는 보이지 않던 문제가 드러납니다.

IA, Journey, Flow의 차이

세 개념은 비슷해 보이지만 보는 대상이 다릅니다.

구분	보는 것	핵심 질문
IA	정보와 기능의 구조	무엇이 어디에 있어야 하는가?
User Journey	사용자의 전체 경험	사용자는 어떤 상황과 감정을 지나가는가?
User Flow	서비스 안의 이동 경로	목표 달성까지 어떤 화면과 행동을 거치는가?

IA는 지도, Journey는 여행 경험, Flow는 실제 이동 경로에 가깝습니다.

User Flow의 기본 표현

User Flow는 복잡한 도구 없이도 다음 요소로 표현할 수 있습니다.

요소	의미
시작	사용자가 흐름에 들어오는 지점
화면	사용자가 보는 화면
행동	클릭, 입력, 선택, 확인
판단	조건에 따라 갈라지는 지점
결과	사용자가 도달하는 상태
예외	오류, 취소, 재시도, 빈 결과

중요한 것은 예쁜 도형보다 사용자가 실제로 어떤 선택을 하는지 보이는 것입니다.

User Flow 작성 순서

User Flow는 다음 순서로 작성합니다.

1. 사용자의 목표를 하나 정한다
2. 시작 지점을 정한다
3. 목표 달성까지 필요한 화면과 행동을 순서대로 적는다
4. 사용자가 선택해야 하는 지점을 표시한다
5. 실패, 취소, 재시도 흐름을 추가한다
6. 마지막 결과 상태를 명확히 쓴다
7. 불필요한 단계가 없는지 줄인다

한 번에 전체 서비스를 그리기보다

하나의 핵심 목표부터 그리는 것이 좋습니다.

User Flow 예시

목표: 사용자가 오늘 점심 메뉴를 추천받는다.

```
흐름
→ 추천 시작
→ 조건 입력
→ 알레르기 입력 여부 확인
→ 추천 생성
→ 결과 화면
→ 추천 이유 확인
→ 저장 또는 다시 추천
```

이 흐름에서 중요한 것은 “추천 생성”보다

사용자가 조건을 이해하고 결과를 판단하는 과정입니다.

판단 지점이 있는 Flow

AI 서비스에는 조건에 따라 갈라지는 지점이 자주 생깁니다.

조건 입력

→ 필수 정보가 충분한가?
→ 예 : 추천 생성
→ 아니오 : 부족한 정보 안내

→ 추천 결과가 안전한가?
→ 예 : 결과 표시
→ 아니오 : 대체 결과 또는 안내 표시

판단 지점을 적으면 서비스가 사용자를 어떻게 보호하는지 보입니다.

AI 서비스 Flow에서 꼭 봐야 할 흐름

AI 기능이 있는 서비스에서는 다음 흐름을 따로 확인해야 합니다.

- 입력 정보가 부족할 때
- AI 결과가 사용자의 조건과 맞지 않을 때
- 결과 신뢰도가 낮을 때
- 민감하거나 위험한 답변이 나올 수 있을 때
- 사용자가 결과를 수정하고 싶을 때
- 사용자가 왜 이런 결과가 나왔는지 알고 싶을 때
- 같은 요청을 다시 시도하고 싶을 때

AI 서비스의 흐름은 성공 경로만으로 충분하지 않습니다.

좋은 User Flow의 기준

좋은 User Flow는 다음 조건을 만족합니다.

- 시작과 끝이 분명하다
- 사용자의 목표가 하나로 명확하다
- 각 단계의 행동이 구체적이다
- 선택 지점이 빠져 있지 않다
- 실패와 되돌아가기 흐름이 있다
- 사용자가 다음 행동을 예측할 수 있다
- AI 결과 이후의 확인, 수정, 저장 흐름이 있다

Flow는 화면 수를 늘리기 위한 문서가 아니라
사용자의 길을 짧고 분명하게 만드는 도구입니다.

Flow에서 자주 생기는 문제

다음 문제는 실제 사용성을 떨어뜨립니다.

- 시작 지점은 있는데 끝 상태가 없다
- 입력 화면이 너무 길다
- 사용자가 왜 입력해야 하는지 모른다
- AI 결과가 나왔지만 다음 행동이 없다
- 오류가 나면 처음부터 다시 해야 한다
- 뒤로 가기, 수정, 취소 흐름이 없다
- 로그인이나 설정이 핵심 행동보다 먼저 강하게 요구된다

사용자는 목표를 이루고 싶지, 절차를 통과하고 싶은 것이 아닙니다.

IA와 Flow를 함께 점검하기

IA와 Flow는 따로 작성해도 마지막에는 함께 봐야 합니다.

점검 질문	의미
중요한 기능이 찾기 쉬운 곳에 있는가?	IA 점검
사용자가 목표까지 자연스럽게 이동하는가?	Flow 점검
사용자가 중간에 길을 잃지 않는가?	IA와 Flow 공통

결과를 보고 다음 행동을 할 수 있는가?	Flow 점검
반복 사용 시 다시 찾기 쉬운가?	IA 점검

구조와 흐름이 맞아야 실제 사용 경험이 자연스러워집니다.

AI 기획자의 IA 점검 질문

IA를 볼 때 다음 질문을 던져 봅니다.

- 사용자가 가장 자주 찾는 정보가 앞에 있는가?
- 메뉴 이름이 비전문가도 이해할 수 있는가?
- AI 결과와 결과의 이유가 같은 맥락에서 보이는가?
- 입력 조건, 결과, 수정 기능이 서로 떨어져 있지 않은가?
- 개인정보나 민감 정보 안내가 필요한 위치에 있는가?
- 나중에 기능이 추가되어도 구조가 유지될 수 있는가?

AI 기획자의 Flow 점검 질문

User Flow를 볼 때 다음 질문을 던져 봅니다.

- 사용자의 목표가 한 문장으로 설명되는가?
- 목표까지 필요한 단계가 너무 많지 않은가?
- AI가 처리하는 동안 사용자는 무엇을 보게 되는가?
- 결과가 마음에 들지 않을 때 다시 시도할 수 있는가?
- 잘못된 입력이나 빈 결과가 나왔을 때 안내가 있는가?
- 사용자가 결과를 저장하거나 비교할 수 있는가?
- 결과를 믿기 위한 정보가 흐름 안에 있는가?

IA와 User Flow를 연결한 예시

식단 추천 서비스의 핵심 구조와 흐름을 연결해 봅니다.

IA 영역	User Flow에서의 역할
홈	추천 시작 지점
조건 입력	개인화 추천을 위한 정보 수집
추천 결과	AI 결과와 이유 확인
결과 조정	조건 수정, 다시 추천
저장 목록	마음에 드는 결과 보관
설정	반복 사용을 위한 기본 조건 관리

IA는 공간을 만들고, Flow는 그 공간을 지나가는 길을 만듭니다.

기획자가 설명할 수 있어야 하는 것

IA와 User Flow를 설계한 뒤에는 다음을 말할 수 있어야 합니다.

- 왜 이 메뉴 구조가 사용자에게 자연스러운가
- 왜 이 정보가 이 위치에 있어야 하는가
- 왜 이 흐름이 핵심 목표에 가장 짧은가
- 어디서 사용자가 불안하거나 막힐 수 있는가
- AI 결과를 사용자가 어떻게 확인하고 수정하는가
- 실패 상황에서 사용자를 어떻게 다시 흐름으로 돌려보내는가

설계의 목적은 문서를 만드는 것이 아니라
팀이 같은 사용자 경험을 상상하게 만드는 것입니다.

오늘의 정리

- IA는 정보와 기능을 사용자가 이해하기 쉬운 구조로 정리하는 일이다
- User Journey는 사용자의 상황, 행동, 감정, Pain Point를 시간 순서로 보는 방법이다
- User Flow는 사용자가 목표를 달성하기 위해 서비스 안에서 이동하는 구체적인 경로다
- AI 서비스에서는 결과 설명, 수정, 재시도, 예외 흐름이 특히 중요하다
- 좋은 설계는 기능을 많이 보여주는 것이 아니라 사용자가 길을 잃지 않게 돕는 것이다

마지막으로 기억할 문장

서비스 구조는 회사가 만든 기능의 목록이 아닙니다.

사용자가 자기 문제를 해결하기 위해
이해하고, 선택하고, 이동하는 길입니다.

예제영상> 카카오품프예약 챗봇 역기획으로 배우는 IA 설계 방법

- 목표설정, 정보나열, 그룹화, 분류
- IA를 다양한 산출물로 확장하기
- 인포메이션 아키텍처를 설계하는 목적

Day 03 — 화면설계서

자료 출처: 화면설계서.pdf

화면설계서(UI Specification)란?

기획자가 기획한 서비스 화면이 어떤 형태로, 어떤 기능을 제공하는지를 상세히 적은 문서이다. 화면을 어떻게 표현할지를 그려 놓은 **와이어프레임**과, 기능을 구현하기 위해 작성하는 **기능 정의서**를 합친 산출물이다. 워터폴·애자일 방법론에 따라 형태 차이는 있지만, 원하는 서비스의 모습을 협업자에게 전달하기 위한 기획자의 최종 산출물이자 그 자체가 커뮤니케이션의 핵심이다.

화면설계서에 들어가야 하는 내용

1. 와이어프레임 & 디스크립션(기능명세서)

- **와이어프레임**: 화면의 레이아웃과 기능 요소를 미리 설계한 초안
- **디스크립션(기능명세서)**: 와이어프레임에 숫자를 표시하여 해당 숫자에 해당하는 설명이나 로직, 디자인 솔루션 사용 등을 설명

예시 구성: Page Title / Screen ID / Author / Date / Screen Path 정보를 상단에 표기하고, 화면 각 요소에 번호(①②③...)를 매긴 뒤 우측 Description 표에 번호별 설명을 기재. 하단에는 Check Point(예: “더보기 누르면 각 해당 페이지로 이동함”)를 별도로 명시한다.

2. 수정 이력 관리

표 형태로 수정된 날짜와 버전명(예: 0.1, 0.2, 1.0), 수정 내용(Description), 수정자(Author)를 기록한다.

3. 메뉴 트리

전체 서비스의 메뉴 구조를 트리 형태로 표현한다. (예: 로그인 → 대시보드 / 식품 데이터 관리 / 고객센터 / 사용자 관리 / 관리자 설정, 각 대메뉴 아래 세부 메뉴를 배치)

4. 스크린 리스트

- 해당 프로젝트에 포함되어 수정되거나 신규로 개발되는 화면 목록
- API 등 화면에 없는 형태로 개발되는 목록

구성: 대메뉴 / 중메뉴 / Screen ID / Page Title / Description / 비교(Related ID) 컬럼으로 정리한 표.

5. 프로세스 플로우 차트

- 서비스의 사용자별 혹은 케이스별 프로세스 플로우
- 화면적인 구분뿐 아니라 데이터적인 로직도 포함

예: 아이디 찾기/비밀번호 찾기 시나리오를 인증방식 선택(이메일 인증/휴대폰 인증) → 인증번호 발송 → 유효시간 내 인증 확인 → 완료까지 분기(Y/N)를 포함한 순서도로 표현.

6. 정책

대부분/중부분별로 서비스 정책을 상세 기술한다. 예: 회원가입 정책의 국가·주소·기본언어·연락처·통화·타임존 등에 대해 표기 가능 범위와 케이스(Case 1/2/3)를 정의하고 최종 결정안을 비교에 명시.

7. 권한 설정

대분류/중분류별로 총관리자·관리자 등 역할별 보기/수정/삭제/쓰기 권한을 O/X로 표로 정리한다.

8. 검증

- **필수 입력항목**: 상황(예: 제품 코드 미입력 시) / 문구(예: “제품코드를 입력해 주세요”) / 처리 시나리오(예: 제품코드 필드로 커서 이동)를 표로 정리
- **유효성 체크**: 상황(예: 출하요청일을 과거로 선택했을 경우) / 문구 / 처리 시나리오(팝오버 띄움, 날짜 자동 세팅 등)를 표로 정리

Day 04 — 서비스시나리오

자료 출처: 서비스_시나리오_및_예외_처리_설계.md

서비스 시나리오 및 예외 처리 설계

AI 서비스가 정상 상황과 문제 상황 모두에서 사용자를 안내하는 방법

오늘의 핵심 질문

- 사용자는 어떤 상황에서 서비스를 사용하게 될까?
- 사용자가 목표를 달성하는 정상 흐름은 어떻게 이어질까?
- 사용자가 막히거나 실패하는 상황은 어디서 생길까?
- 오류가 발생했을 때 어떤 문구와 행동 안내가 필요할까?
- AI 결과가 기대와 다를 때 서비스는 어떻게 대응해야 할까?

기능이 작동하는 것과 서비스가 완성되는 것은 다르다

기능이 작동해도 사용자가 막히면 서비스 경험은 끊깁니다.

예를 들어 추천 기능이 있어도 다음 상황을 생각해야 합니다.

- 사용자가 조건을 잘못 입력하면?
- 추천할 수 있는 결과가 없으면?
- AI가 너무 일반적인 답변을 하면?
- 사용자가 결과를 수정하고 싶으면?
- 네트워크 문제로 결과가 늦게 나오면?

AI 기획자는 “성공하는 화면”뿐 아니라

사용자가 막히는 순간의 안내와 회복 흐름까지 설계해야 합니다.

User Scenario란 무엇인가

User Scenario는 사용자가 특정 상황에서 서비스를 이용해 목표를 달성하는 과정을 이야기처럼 정리한 것입니다.

User Scenario는 다음을 함께 설명합니다.

- 사용자는 누구인가
- 어떤 상황에 놓여 있는가
- 어떤 목표를 가지고 있는가
- 어떤 행동을 순서대로 하는가
- 어떤 정보와 기능을 만나는가
- 어디서 만족하거나 막힐 수 있는가

즉, User Scenario는 기능을 실제 사용 상황으로 바꾸는 문서입니다.

페르소나 프로파일은 특정 사용자 또는 사용자 그룹을 상세하게 묘사하는 도구이며 사용자 시나리오 작성 및 제품 디자인에서 중요한 역할을 합니다.

[원본 슬라이드 이미지 - 참고용, 본 문서에는 미포함]

alt text

User Scenario가 필요한 이유

기능 목록만 보면 서비스가 너무 이상적으로 보입니다.

하지만 시나리오로 보면 현실적인 질문이 생깁니다.

- 사용자가 처음에 무엇을 보고 시작하는가?

- 이 입력을 왜 해야 하는지 이해하는가?
- 결과가 나오기 전까지 기다릴 수 있는가?
- 결과가 마음에 들지 않을 때 무엇을 할 수 있는가?
- 오류가 났을 때 사용자가 다시 시도할 수 있는가?

시나리오는 기능의 빈틈을 발견하게 해 줍니다.

User Story와 User Scenario의 차이

두 개념은 연결되어 있지만 역할이 다릅니다.

구분	설명	예시
User Story	사용자의 필요와 가치를 짧게 표현	나는 식단 관리자로서 알레르기를 반영한 메뉴를 추천받고 싶다
User Scenario	실제 사용 상황과 행동 흐름을 구체적으로 설명	점심시간 전 사용자가 조건을 입력하고 추천 결과를 확인한 뒤 저장한다

User Story는 “왜 필요한가”를 말하고,
User Scenario는 “어떻게 사용되는가”를 보여 줍니다.

User Flow와 User Scenario의 차이

User Flow는 화면과 행동의 이동 경로를 보여 줍니다.

User Scenario는 그 흐름이 일어나는 상황과 맥락을 함께 설명합니다.

구분	중심 질문
User Flow	사용자는 어떤 화면과 행동을 거치는가?
User Scenario	사용자는 어떤 상황에서 왜 그렇게 행동하는가?

Flow가 길이라면 Scenario는 그 길을 걷는 사용자의 상황입니다.

좋은 User Scenario의 구성

좋은 User Scenario에는 다음 요소가 들어갑니다.

요소	질문
사용자	누가 사용하는가?
상황	언제, 어떤 문제 때문에 사용하는가?
목표	사용자가 얻고 싶은 결과는 무엇인가?

요소	질문
시작점	어디서 서비스를 시작하는가?
행동	어떤 순서로 움직이는가?
결과	마지막에 어떤 상태가 되는가?
감정	어디서 안심하거나 불안해하는가?
예외	어디서 막힐 수 있는가?

시나리오는 기능 설명이 아니라 사용자의 경험 설명입니다.

User Scenario 예시

건강 식단 추천 서비스를 예로 들어 봅니다.

점심 메뉴를 고민하는 직장인이 점심시간 20분 전에 서비스를 연다.
사용자는 알레르기과 선호 음식을 입력하고 오늘의 식단 추천을 요청한다.
서비스는 추천 메뉴와 추천 이유, 주의할 재료를 함께 보여 준다.
사용자는 마음에 드는 메뉴를 저장하거나 조건을 조금 바꿔 다시 추천받는다.

이 시나리오에는 사용자, 상황, 목표, 행동, 결과가 모두 들어 있습니다.

시나리오가 너무 추상적일 때

약한 시나리오:

사용자는 AI 추천 서비스를 사용한다.

이 문장은 실제 설계에 도움이 되기 어렵습니다.

왜냐하면 다음 정보가 없기 때문입니다.

- 어떤 사용자인가
- 어떤 문제 상황인가
- 왜 추천이 필요한가
- 무엇을 입력하는가
- 결과를 보고 무엇을 하는가
- 실패하면 어떻게 되는가

시나리오는 구체적일수록 설계 판단에 도움이 됩니다.

시나리오가 너무 세부적일 때

반대로 시나리오가 너무 작은 동작까지 설명하면 읽기 어렵습니다.

예를 들어:

- 버튼 색상
- 아이콘 위치
- 문구의 모든 띄어쓰기
- 화면 좌표
- 세부 애니메이션

이런 내용은 화면 설계에서 다룰 수 있습니다.

User Scenario에서는 사용자의 상황, 목표, 행동, 판단 흐름에 집중합니다.

User Scenario 작성 기준

AI 기획자는 시나리오를 쓸 때 다음 기준을 봅니다.

- 실제 사용자가 겪을 만한 상황인가?
- 사용자의 목표가 분명한가?
- 서비스의 핵심 기능이 자연스럽게 등장하는가?
- 사용자가 결과를 이해하고 다음 행동을 할 수 있는가?
- 막히는 지점이 함께 보이는가?
- AI가 판단하는 부분과 사용자가 판단하는 부분이 구분되는가?

좋은 시나리오는 정상 흐름과 예외 흐름의 출발점이 됩니다.

예외 처리 흐름

[페이상품권 서버 개발 - 예외 처리 예시](#)

[원본 슬라이드 이미지 - 참고용, 본 문서에는 미포함]

alt text

정상 흐름이란 무엇인가

정상 흐름은 사용자가 의도한 대로 서비스를 사용해 목표를 달성하는 기본 경로입니다.

다른 말로는 주요 흐름, 기본 흐름, 성공 흐름이라고도 볼 수 있습니다.

정상 흐름은 다음을 보여 줍니다.

- 사용자가 어디서 시작하는가
- 어떤 정보를 입력하는가
- 서비스가 어떤 결과를 제공하는가
- 사용자가 결과를 어떻게 확인하는가
- 마지막에 어떤 상태에 도달하는가

정상 흐름은 서비스 경험의 중심 줄기입니다.

정상 흐름 예시

목표: 사용자가 오늘 먹을 수 있는 점심 메뉴를 추천받는다.

홈 화면 진입
→ 추천 시작 선택
→ 알레르기 및 선호 음식 입력
→ 식사 시간과 예산 선택
→ 추천 요청
→ 추천 결과 확인
→ 추천 이유와 주의사항 확인
→ 마음에 드는 메뉴 저장

이 흐름은 사용자가 막히지 않고 목표에 도달하는 기본 경로입니다.

정상 흐름을 작성할 때 필요한 질문

정상 흐름을 작성할 때는 다음을 확인합니다.

- 사용자의 시작점이 명확한가?
- 사용자가 무엇을 입력해야 하는가?
- 입력 이유를 사용자가 이해할 수 있는가?
- 결과가 나오기까지 기다리는 과정이 있는가?
- 결과 화면에서 핵심 정보가 먼저 보이는가?
- 사용자의 마지막 행동이 명확한가?

정상 흐름은 단순히 화면을 나열하는 것이 아닙니다.

사용자가 목표를 달성하는 과정을 설명하는 것입니다.

정상 흐름만 있으면 부족한 이유

현실의 사용자는 항상 정상 흐름으로 움직이지 않습니다.

다음과 같은 상황이 자주 생깁니다.

- 필수 정보를 입력하지 않는다
- 너무 모호한 요청을 한다
- 조건이 서로 충돌한다
- 원하는 결과가 없다
- AI 결과가 부정확하거나 위험할 수 있다 ..

그래서 정상 흐름 다음에는 반드시 예외 흐름을 설계해야 합니다.

예외 흐름이란 무엇인가

예외 흐름은 사용자가 목표를 달성하는 과정에서 문제가 생겼을 때의 대응 경로입니다.

예외 흐름은 오류만 의미하지 않습니다.

- 입력이 부족한 상황
- 조건이 맞지 않는 상황
- 결과가 없는 상황
- 사용자가 다른 선택을 하는 상황
- 시스템이 응답하지 않는 상황
- AI가 답변을 제한해야 하는 상황

예외 흐름의 목적은 사용자를 실패 상태에 버려두지 않는 것입니다.

대안 흐름과 예외 흐름

흐름을 볼 때는 대안 흐름과 예외 흐름을 구분하면 좋습니다.

구분	의미	예시
정상 흐름	가장 기본적인 성공 경로	조건 입력 후 추천 결과 확인
대안 흐름	사용자가 다른 선택을 해도 목표에 갈 수 있는 경로	조건을 수정하고 다시 추천
예외 흐름	문제가 발생해 안내와 회복이 필요한 경로	필수 정보 누락, 결과 없음, AI 응답 실패

대안 흐름은 선택의 문제이고, 예외 흐름은 회복의 문제입니다.

예외 흐름을 찾는 방법

예외 흐름은 각 단계마다 질문을 던져 찾습니다.

단계	예외 질문
시작	사용자가 로그인하지 않았거나 접근 권한이 없으면?
입력	필수 정보를 빠뜨리거나 잘못 입력하면?
처리	결과 생성이 오래 걸리거나 실패하면?
결과	보여줄 결과가 없거나 조건과 맞지 않으면?
확인	사용자가 결과를 믿지 못하거나 수정하고 싶으면?
저장	저장에 실패하거나 이미 저장된 결과라면?

예외 흐름은 정상 흐름의 각 단계에서 파생됩니다.

AI 서비스에서 자주 생기는 예외

AI 서비스에서는 다음 예외를 특히 주의해야 합니다.

예외 상황	사용자가 느끼는 문제
입력이 모호함	무엇을 더 써야 하는지 모른다
데이터가 부족함	왜 결과가 없는지 이해하기 어렵다
결과 신뢰도가 낮음	추천을 믿어도 되는지 불안하다
위험하거나 민감한 요청	서비스가 어디까지 도와줄 수 있는지 모른다
응답 지연	멈춘 것처럼 느낀다
시스템 오류	다시 시도해야 하는지 알 수 없다

AI 예외는 기술 문제가 아니라 사용자 신뢰의 문제입니다.

예외 흐름 설계 순서

예외 흐름은 다음 순서로 설계합니다.

1. 정상 흐름의 각 단계를 나눈다
2. 각 단계에서 실패하거나 다른 선택을 하는 상황을 찾는다

3. 사용자가 무엇을 알 필요가 있는지 정리한다
4. 사용자가 다음에 할 수 있는 행동을 정한다
5. 오류 메시지와 버튼 문구를 연결한다
6. 사용자가 정상 흐름으로 돌아갈 수 있는 길을 만든다
7. 반복해서 같은 오류가 나지 않도록 입력이나 구조를 조정한다

좋은 예외 흐름은 사용자를 다시 앞으로 움직이게 합니다.

예외 흐름 예시: 필수 정보 누락

상황:

사용자가 알레르기 정보를 입력하지 않고 식단 추천을 요청한다.

흐름:

- 조건 입력
- 추천 요청
- 필수 정보 누락 확인
- 누락된 항목 안내
- 사용자가 알레르기 정보 입력
- 다시 추천 요청
- 추천 결과 표시

핵심은 “실패”를 보여주는 것이 아니라
무엇을 채우면 계속할 수 있는지 알려주는 것입니다.

예외 흐름 예시: 결과 없음

상황:

사용자의 조건이 너무 좁아 추천 가능한 메뉴가 없다.

흐름:

- 추천 요청
- 조건에 맞는 결과 없음
- 결과가 없는 이유 안내
- 완화할 수 있는 조건 제안
- 조건 수정 또는 다시 입력
- 추천 결과 재생성

결과 없음은 막다른 길이 아닙니다.
서비스가 다음 선택지를 제안해야 합니다.

예외 흐름 예시: AI 응답 제한

상황:

사용자가 건강에 위험할 수 있는 단정적인 의학 조언을 요청한다.

흐름:

- 사용자 요청 입력
- 민감하거나 위험한 요청 확인
- 서비스 제공 범위 안내
- 일반 정보 또는 안전한 대안 안내
- 전문가 상담 권장
- 사용자가 다른 질문으로 전환

AI 서비스는 모든 요청에 답하는 것이 아니라

안전하게 답할 수 있는 범위를 지켜야 합니다.

예외 흐름 예시: AI 응답 실패

상황:

AI 응답 생성 중 일시적인 오류가 발생한다.

흐름:

추천 요청
→ 응답 생성 실패
→ 일시적인 문제 안내
→ 다시 시도 버튼 제공
→ 입력 내용 유지
→ 사용자가 다시 요청

중요한 것은 사용자가 입력한 내용을 잃지 않게 하는 것입니다.

예외 흐름에서 피해야 할 대응

다음 대응은 사용자의 불안을 키웁니다.

- “오류가 발생했습니다”만 보여준다
- 사용자가 무엇을 잘못했는지 모호하게 말한다
- 처음 화면으로 강제로 돌려보낸다
- 입력한 내용을 모두 지운다
- 다음 행동을 제시하지 않는다
- 기술 용어와 오류 코드만 보여준다
- AI가 못 하는 일을 사용자가 이해하지 못하게 한다

예외 흐름의 핵심은 안내, 선택지, 회복입니다.

오류 메시지 설계란 무엇인가

오류 메시지 설계는 문제가 생겼을 때 사용자에게 상황과 다음 행동을 알려주는 문구를 설계하는 일입니다.

오류 메시지는 단순한 경고 문구가 아닙니다.

좋은 오류 메시지는 다음을 합니다.

- 무슨 일이 생겼는지 알려준다
- 왜 진행이 어려운지 설명한다
- 사용자가 다음에 할 행동을 제시한다
- 사용자의 불안과 책임감을 줄인다
- 서비스 신뢰를 유지한다

좋은 오류 메시지의 구성

오류 메시지는 보통 네 가지 요소로 생각할 수 있습니다.

요소	질문
상황	지금 무슨 일이 생겼는가?
이유	왜 진행할 수 없는가?
행동	사용자가 무엇을 하면 되는가?
안심	입력값이나 상태가 안전하게 유지되는가?

모든 메시지에 네 가지가 다 길게 들어갈 필요는 없습니다.

하지만 사용자가 다음 행동을 알 수 있어야 합니다.

나쁜 오류 메시지와 좋은 오류 메시지

나쁜 메시지	좋은 메시지
오류가 발생했습니다	추천 결과를 불러오지 못했습니다. 잠시 후 다시 시도해 주세요.
잘못된 입력입니다	알레르기 항목을 선택해야 추천을 시작할 수 있습니다.
결과 없음	조건에 맞는 메뉴가 없습니다. 예산이나 제외 음식을 조정해 보세요.
접근 불가	저장한 추천을 보려면 로그인이 필요합니다.
AI 응답 실패	일시적으로 답변을 만들지 못했습니다. 입력 내용은 유지되어 있습니다.

좋은 메시지는 문제보다 해결 방법을 먼저 생각하게 합니다.

오류 메시지의 말투

오류 메시지는 짧고 구체적이어야 합니다.

좋은 말투의 기준:

- 사용자를 탓하지 않는다
- 기술 용어를 줄인다
- 다음 행동을 동사로 안내한다
- 불필요하게 무섭게 말하지 않는다
- 같은 상황에서는 같은 표현을 쓴다
- 중요한 정보는 숨기지 않는다

예외 상황일수록 서비스의 말투가 신뢰를 만듭니다.

오류 메시지에서 피해야 할 표현

다음 표현은 사용자를 더 혼란스럽게 만들 수 있습니다.

피해야 할 표현	이유
잘못하셨습니다	사용자를 탓하는 느낌을 준다
알 수 없는 오류	다음 행동을 알기 어렵다
시스템 에러 500	비전문가에게 의미가 없다
다시 하세요	무엇을 다시 해야 하는지 모호하다
사용 불가	왜 불가능한지, 대안이 무엇인지 모른다

오류 메시지는 내부 시스템의 말을 사용자 언어로 번역해야 합니다.

AI 서비스 오류 메시지에서 중요한 점

AI 서비스의 오류 메시지는 특히 신뢰와 안전을 고려해야 합니다.

- AI가 할 수 없는 일을 솔직하게 말한다
- 결과가 제한된 이유를 쉬운 말로 설명한다
- 사용자가 수정할 수 있는 입력 조건을 알려준다
- 민감한 상황에서는 단정적인 답변을 피한다
- 입력 내용이 유지되는지 알려준다
- 필요하면 사람의 판단이나 전문가 상담을 안내한다

AI 오류 메시지는 단순한 실패 안내가 아니라 서비스의 책임 범위를 보여 줍니다.

상황별 오류 메시지 예시

상황	메시지 예시
필수 입력 누락	알레르기 정보를 선택해야 추천을 시작할 수 있습니다.
조건 충돌	선택한 조건을 모두 만족하는 메뉴를 찾지 못했습니다. 제외 음식을 줄여 보세요.

상황	메시지 예시
결과 없음	지금 조건에 맞는 추천 결과가 없습니다. 예산이나 식사 시간을 조정해 보세요.
응답 지연	추천을 준비하는 데 시간이 조금 더 걸리고 있습니다. 잠시만 기다려 주세요.
AI 응답 실패	일시적으로 추천을 만들지 못했습니다. 입력 내용은 유지되어 있습니다.
안전 제한	이 요청은 안전을 위해 답변 범위가 제한됩니다. 일반적인 정보로 다시 안내해 드릴게요.

메시지는 상황과 다음 행동이 함께 보여야 합니다.

오류 메시지와 버튼은 함께 설계한다

오류 메시지만 있고 행동 버튼이 없으면 사용자는 멈춥니다.

오류 상황	메시지 이후 필요한 행동
필수 입력 누락	입력하러 가기
결과 없음	조건 수정하기
응답 실패	다시 시도하기
로그인 필요	로그인하기
저장 실패	다시 저장하기
안전 제한	질문 바꿔보기

문구와 버튼은 한 쌍으로 설계해야 합니다.

오류 메시지 위치도 중요하다

오류 메시지는 사용자가 문제를 해결할 수 있는 위치에 있어야 합니다.

- 입력 오류는 해당 입력 항목 가까이에 보여 준다
- 전체 처리 실패는 결과 영역이나 화면 상단에 보여 준다
- 저장 실패는 저장 버튼 근처에 보여 준다
- AI 답변 제한은 답변이 나오는 영역에 보여 준다
- 반복 오류는 도움말이나 고객센터 경로를 함께 보여 준다

메시지가 멀리 있으면 사용자는 무엇을 고쳐야 하는지 모릅니다.

정상 흐름과 예외 흐름을 함께 보는 방법

정상 흐름을 먼저 쓰고, 각 단계 옆에 예외를 붙여 봅니다.

정상 단계	가능한 예외	대응
조건 입력	필수 정보 누락	누락 항목 안내
추천 요청	응답 지연	진행 상태 표시
결과 표시	결과 없음	조건 수정 제안
결과 확인	신뢰 부족	추천 이유 제공
저장	저장 실패	다시 시도 안내

이렇게 보면 서비스가 어디서 사용자를 놓칠 수 있는지 보입니다.

시나리오 설계에서 AI 기획자가 봐야 할 것

AI 기획자는 다음을 특히 확인해야 합니다.

- AI가 답하는 지점과 사용자가 판단하는 지점이 구분되는가?
- AI 결과를 설명하는 정보가 충분한가?
- 사용자가 결과를 수정하거나 다시 요청할 수 있는가?
- 결과가 없을 때 대안 행동이 있는가?

- 민감하거나 위험한 요청을 제한하는 흐름이 있는가?
- 오류가 나도 입력 내용과 사용자의 맥락이 유지되는가?
- 메시지가 사용자를 탓하지 않고 다음 행동을 알려 주는가?

서비스 시나리오 점검 질문

User Scenario를 다 쓴 뒤에는 다음을 확인합니다.

- 사용자 상황이 실제적인가?
- 사용자의 목표가 분명한가?
- 정상 흐름의 시작과 끝이 명확한가?
- 각 단계에서 필요한 정보와 화면이 연결되는가?
- 사용자가 막힐 만한 지점이 빠져 있지 않은가?
- 예외 흐름이 정상 흐름으로 돌아갈 길을 제공하는가?
- 오류 메시지가 사용자의 다음 행동을 안내하는가?

시나리오를 한 번 쓰고 끝나는 문서가 아니라 설계를 점검하는 도구입니다.

오늘의 정리

- User Scenario는 사용자의 상황, 목표, 행동, 결과를 이야기처럼 정리하는 방법이다
- 정상 흐름은 사용자가 목표를 달성하는 기본 성공 경로다
- 예외 흐름은 입력 누락, 결과 없음, AI 응답 실패처럼 사용자가 막히는 상황의 회복 경로다
- 오류 메시지는 문제 상황, 이유, 다음 행동을 사용자 언어로 안내해야 한다
- AI 서비스에서는 예외 처리와 오류 메시지가 서비스 신뢰를 만드는 중요한 설계 요소다

마지막으로 기억할 문장

좋은 서비스는 사용자가 성공할 때만 친절한 것이 아닙니다.

**사용자가 막혔을 때
무슨 일이 생겼는지 알려 주고,
다시 앞으로 갈 수 있는 길을 보여 줍니다.**

Day 05 — 웹_프로토타입구현

자료 출처: 프로토타입_구현_및_검증.md

프로토타입 구현 및 검증

AI 서비스 아이디어를 빠르게 확인하고 개선하는 방법

오늘의 핵심 질문

- 프로토타입은 완성된 서비스와 무엇이 다를까?
- 프로토타입을 만드는 진짜 목적은 무엇일까?
- 사용자는 프로토타입을 보며 어떤 반응을 보여 줄까?
- 피드백은 어떻게 모으고, 어떻게 다음 버전에 반영해야 할까?
- AI 기능이 있는 프로토타입은 무엇을 더 조심해서 검증해야 할까?

프로토타입이란 무엇인가

프로토타입은 완성된 서비스를 만들기 전에 핵심 경험을 빠르게 확인하기 위해 만든 임시 모델입니다.

프로토타입은 다음을 보여 줍니다.

- 사용자가 어떤 화면을 보게 되는지
- 어떤 순서로 이동하는지
- 어떤 정보를 입력하는지
- 어떤 결과를 받는지
- 어디서 이해하거나 막히는지

프로토타입은 완성품이 아니라 **검증을 위한 대화 도구**입니다.

[원본 슬라이드 이미지 - 참고용, 본 문서에는 미포함]

alt text

프로토타입은 왜 필요한가

기획 문서만으로는 사용자의 반응을 알기 어렵습니다.

문서에서는 자연스러워 보였던 흐름도 실제로 보여 주면 다르게 보일 수 있습니다.

- 사용자가 메뉴 이름을 이해하지 못한다
- 버튼을 눌러야 하는지 모른다
- 입력 항목이 부담스럽다고 느낀다
- AI 결과를 신뢰하지 않는다
- 오류 메시지를 보고도 무엇을 해야 할지 모른다

프로토타입은 이런 문제를 일찍 발견하게 해 줍니다.

프로토타입의 목적

프로토타입의 목적은 멋진 화면을 자랑하는 것이 아닙니다.

핵심 목적은 다음과 같습니다.

목적	설명
이해 확인	사용자가 화면과 문구를 이해하는지 본다
흐름 확인	목표까지 자연스럽게 이동하는지 본다
가치 확인	사용자가 결과를 유용하다고 느끼는지 본다

위험 확인	막히는 지점과 불안 요소를 찾는다
의사결정	무엇을 유지하고 바꿀지 정한다

프로토타입은 “만들었다”보다 “무엇을 알게 되었는가”가 중요합니다.

프로토타입으로 확인해야 할 것

AI 기획자는 프로토타입을 통해 다음을 확인해야 합니다.

- 사용자가 서비스의 목적을 바로 이해하는가?
- 첫 화면에서 무엇을 해야 할지 아는가?
- 입력 항목이 너무 어렵거나 많지 않은가?
- AI 결과가 사용자의 기대와 맞는가?
- 추천 이유나 답변 근거가 충분한가?
- 결과가 마음에 들지 않을 때 다시 시도할 수 있는가?
- 오류 상황에서 다음 행동이 보이는가?

프로토타입은 기능 수보다 사용자 판단을 확인하는 데 초점을 둡니다.

프로토타입이 아닌 것

프로토타입을 완성품처럼 생각하면 목적이 흐려집니다.

오해	바르게 보기
모든 기능이 작동해야 한다	핵심 흐름만 확인해도 된다
디자인이 완벽해야 한다	이해와 흐름 검증이 먼저다
개발이 끝나야 보여줄 수 있다	빠르게 보여주고 빨리 배운다
피드백은 출시 후 받는다	출시 전에도 충분히 배울 수 있다
사용자가 좋아하면 끝이다	왜 좋아하는지, 어디서 막히는지 봐야 한다

프로토타입은 완성도를 증명하는 문서가 아니라 가설을 확인하는 도구입니다.

프로토타입의 종류

프로토타입은 정교함에 따라 여러 단계로 나눌 수 있습니다.

구분	특징	확인하기 좋은 것
낮은 완성도	손그림, 간단한 와이어프레임	구조와 흐름
중간 완성도	클릭 가능한 화면	이동 경로와 문구 이해
높은 완성도	실제 서비스처럼 보이는 화면	사용감과 세부 반응
기능형	일부 기능이 실제로 작동	데이터, AI 결과, 처리 흐름

항상 높은 완성도가 좋은 것은 아닙니다.

검증하려는 질문에 맞는 수준이면 충분합니다.

낮은 완성도 프로토타입

낮은 완성도 프로토타입은 빠르게 구조를 확인할 때 좋습니다.

예를 들어:

- 종이에 그린 화면
- 간단한 와이어프레임
- 메뉴 구조 그림
- 클릭은 되지 않지만 흐름을 설명할 수 있는 화면

장점은 빠르고 수정이 쉽다는 것입니다.

아직 큰 방향이 정해지지 않았다면 낮은 완성도로 먼저 확인하는 것이 좋습니다.

높은 완성도 프로토타입

높은 완성도 프로토타입은 실제 서비스에 가까운 느낌을 확인할 때 좋습니다.

예를 들어:

- 실제 색상과 문구가 들어간 화면
- 클릭 가능한 화면 전환
- 실제 데이터처럼 보이는 결과
- 로딩, 빈 결과, 오류 상태가 포함된 화면

단점은 만드는 데 시간이 더 걸리고, 사용자가 이미 완성된 서비스라고 오해할 수 있다는 점입니다.

AI 서비스 프로토타입의 특징

AI 서비스 프로토타입은 일반 화면보다 더 확인할 것이 많습니다.

- 사용자가 AI에게 무엇을 기대하는가
- 입력 정보가 충분한가
- AI 결과가 유용하고 이해 가능한가
- 결과가 틀렸을 때 사용자가 알아차릴 수 있는가
- AI가 답하지 않아야 하는 상황이 있는가
- 사용자에게 결과 수정과 재시도 방법이 보이는가

AI 프로토타입은 화면만이 아니라

AI 결과를 사용자가 어떻게 받아들이는지까지 확인해야 합니다.

AI 프로토타입에서 가짜 결과를 써도 되는가

초기 검증에서는 실제 AI가 완전히 작동하지 않아도 됩니다.

다음처럼 보여줄 수 있습니다.

- 미리 준비한 추천 결과
- 사람이 만든 답변 예시
- 실제처럼 보이는 샘플 데이터
- AI가 답변한 것처럼 보이는 흐름

중요한 것은 속이는 것이 아니라, 검증 목적을 분명히 하는 것입니다.

지금은 기술 성능이 아니라
사용자가 이 결과를 이해하고 가치 있다고 느끼는지를 확인하는 단계입니다.

프로토타입 구현 전에 정할 것

프로토타입을 만들기 전에 먼저 검증 질문을 정해야 합니다.

검증 질문	확인 방법
사용자가 첫 화면을 이해하는가?	첫 화면을 보고 무엇을 할지 말하게 한다
입력 항목이 부담스럽지 않은가?	입력 중 망설이는 지점을 본다
추천 결과가 유용한가?	결과를 보고 선택할 수 있는지 본다
이유 설명이 충분한가?	왜 이 결과가 나왔는지 설명할 수 있는지 본다
오류 메시지가 도움이 되는가?	오류 후 다음 행동을 아는지 본다

프로토타입은 질문 없이 만들면 단순한 화면 제작이 됩니다.

좋은 검증 질문의 기준

좋은 검증 질문은 관찰할 수 있어야 합니다.

악한 질문:

사용자가 좋아할까?

좋은 질문:

사용자가 첫 화면을 보고 추천 시작 버튼을 찾을 수 있는가?

악한 질문:

AI 결과가 괜찮을까?

좋은 질문:

사용자가 추천 이유를 보고 결과를 믿을 수 있다고 말하는가?

검증 질문은 사용자의 행동과 판단으로 확인할 수 있어야 합니다.

프로토타입 구현 범위 정하기

프로토타입은 모든 화면을 만들 필요가 없습니다.

우선 다음 범위부터 확인합니다.

- 사용자가 서비스를 시작하는 화면
- 핵심 정보를 입력하는 화면
- AI 결과를 확인하는 화면
- 결과를 수정하거나 다시 시도하는 흐름
- 결과가 없거나 오류가 나는 상태

특히 AI 서비스에서는 성공 화면만 만들면 부족합니다.
신뢰와 회복 흐름을 함께 보여 줘야 합니다.

프로토타입에 포함할 상태

사용자는 여러 상태를 경험합니다.

상태	설명
기본 상태	사용자가 처음 보는 화면
입력 중 상태	사용자가 정보를 작성하는 화면
로딩 상태	AI 결과를 기다리는 화면
성공 상태	결과가 정상적으로 나온 화면
빈 결과 상태	조건에 맞는 결과가 없는 화면
오류 상태	응답 실패나 입력 오류가 발생한 화면

이 상태들을 확인해야 실제 사용 흐름이 끊기지 않습니다.

프로토타입 검증이란 무엇인가

프로토타입 검증은 사용자가 프로토타입을 보거나 사용하면서 목표를 달성할 수 있는지 확인하는 과정입니다.

검증은 다음을 보는 일입니다.

- 사용자가 어디서 멈추는가
- 어떤 표현을 이해하지 못하는가
- 어떤 기능을 기대하는가
- 어떤 결과를 신뢰하지 못하는가

- 어떤 흐름이 불필요하게 길게 느껴지는가

검증은 평가가 아니라 학습입니다.

검증에서 중요한 태도

프로토타입 검증의 목적은 기획안을 방어하는 것이 아닙니다.

중요한 태도는 다음과 같습니다.

- 사용자가 틀렸다고 생각하지 않는다
- 설명하기 전에 먼저 관찰한다
- 좋은 반응보다 막히는 지점을 더 중요하게 본다
- 한 사람의 의견을 전체 결론으로 단정하지 않는다
- 반복해서 나타나는 신호를 찾는다
- 피드백을 기능 추가 요청으로만 해석하지 않는다

사용자의 말보다 행동에서 더 많은 단서를 얻을 수 있습니다.

사용자 피드백이란 무엇인가

사용자 피드백은 사용자가 서비스 경험에 대해 보여 주는 반응과 의견입니다.

피드백에는 여러 종류가 있습니다.

피드백 종류	예시
말로 표현한 의견	이 결과는 조금 믿기 어려워요
행동으로 보인 반응	버튼을 찾지 못하고 화면을 오래 본다
감정 반응	입력 항목에서 부담을 느낀다
기대 표현	결과를 비교해 보고 싶다고 말한다
불편 표현	왜 이 정보를 입력해야 하는지 모르겠다고 한다

좋은 기획자는 사용자의 말과 행동을 함께 봅니다.

피드백

[원본 슬라이드 이미지 - 참고용, 본 문서에는 미포함]

w:1000

피드백을 받을 때의 질문

사용자에게는 정답을 유도하는 질문보다 경험을 말하게 하는 질문이 좋습니다.

좋은 질문:

- 이 화면을 처음 봤을 때 무엇을 해야 한다고 느꼈나요?
- 어떤 부분에서 망설였나요?
- 결과를 보고 어떤 판단을 할 수 있었나요?
- 추천 이유가 충분히 이해되었나요?
- 다시 시도하거나 수정하고 싶을 때 방법이 보였나요?
- 이 서비스가 실제 상황에서 도움이 될 것 같나요?

질문은 사용자의 실제 판단을 끌어내야 합니다.

피해야 할 질문

다음 질문은 좋은 피드백을 얻기 어렵습니다.

피해야 할 질문	이유
괜찮죠?	긍정 답변을 유도한다
예쁘지 않나요?	디자인 취향으로만 흐른다
이 서비스 유용하죠?	피드백을 이끌어내지 못함

이 기능 필요하쇼?	필요하냐고 답하기 쉽나
AI 결과 좋았죠?	실제 신뢰 여부를 알기 어렵다
어떤 기능을 더 넣을까요?	기능 추가 목록만 늘어날 수 있다

좋은 피드백 질문은 사용자가 느낀 문제와 판단 과정을 드러내야 합니다.

피드백을 기록하는 기준

피드백은 그냥 메모하면 나중에 판단하기 어렵습니다.

항목	기록 내용
상황	어떤 화면이나 흐름에서 나온 반응인가
사용자 말	사용자가 실제로 한 말
관찰 행동	사용자가 멈춘 지점, 클릭한 순서
문제 유형	이해 문제, 흐름 문제, 신뢰 문제, 기능 문제
영향도	핵심 목표 달성에 얼마나 큰 영향을 주는가
개선 방향	문구 수정, 구조 변경, 기능 조정 등

피드백은 느낌이 아니라 의사결정 근거가 되어야 합니다.

피드백을 해석할 때 주의할 점

사용자의 말은 중요하지만 그대로 모두 반영하면 안 됩니다.

예를 들어 사용자가 이렇게 말할 수 있습니다.

검색 기능이 있으면 좋겠어요.

이때 바로 검색 기능을 추가하기보다 질문해야 합니다.

- 사용자가 무엇을 찾지 못했는가?
- 메뉴 구조가 헷갈렸는가?
- 결과가 너무 많았는가?
- 추천 결과를 비교하고 싶었던 것인가?

피드백은 요청 그대로가 아니라 그 뒤에 있는 문제를 찾아야 합니다.

피드백 분류하기

피드백은 다음처럼 분류할 수 있습니다.

분류	의미	예시
이해 문제	화면이나 문구를 이해하지 못함	추천 시작 버튼이 보이지 않음
흐름 문제	목표까지 가는 길이 어색함	결과 수정 위치를 찾지 못함
신뢰 문제	결과를 믿기 어려움	추천 이유가 부족함
입력 문제	작성 부담이나 혼란이 큼	알레르기 입력 방식이 어렵다
예외 문제	막혔을 때 회복이 안 됨	결과 없음 후 다음 행동이 없다
가치 문제	결과가 도움이 되지 않음	추천이 너무 일반적이다

분류를 해야 무엇을 먼저 고칠지 보입니다.

피드백 우선순위 정하기

모든 피드백을 한 번에 반영할 수는 없습니다.

우선순위는 다음 기준으로 정합니다.

- 핵심 목표 달성에 직접 영향을 주는가?
- 여러 사용자에게 반복해서 나타나는가?
- 사용자가 중간에 이탈할 정도로 큰 문제인가?
- AI 결과의 신뢰와 안전에 영향을 주는가?
- 작은 수정으로 큰 개선을 만들 수 있는가?
- 다음 검증 전에 반드시 고쳐야 하는가?

피드백 반영도 기능 우선순위처럼 판단 기준이 필요합니다.

사용자 피드백 반영의 기본 흐름

피드백은 다음 순서로 반영합니다.

1. 피드백을 모은다
2. 비슷한 문제끼리 묶는다
3. 문제의 원인을 해석한다
4. 우선순위를 정한다
5. 개선 방향을 결정한다
6. 프로토타입을 수정한다
7. 다시 사용자 반응을 확인한다

피드백 반영은 한 번의 수정이 아니라 반복 과정입니다.

피드백을 반영하는 방식

피드백은 꼭 큰 기능 추가로 이어지지 않습니다.

피드백 문제	반영 방식
버튼을 못 찾음	버튼 위치와 문구 수정
입력 이유를 모름	입력 항목 옆 설명 추가
결과를 못 믿음	추천 이유와 근거 표현 강화
결과가 너무 많음	핵심 결과 먼저 보여주기
오류 후 막힘	다음 행동 버튼 추가
AI 답변이 모호함	답변 형식과 제한 안내 개선

문제에 맞는 가장 작은 개선부터 생각하는 것이 좋습니다.

AI 서비스 피드백에서 특히 봐야 할 것

AI 서비스에서는 다음 피드백이 중요합니다.

- 사용자가 AI 결과를 이해했는가?
- 사용자가 결과를 믿을 수 있다고 느끼는가?
- 결과가 틀릴 수 있다는 점을 받아들일 수 있는가?
- 수정하거나 다시 요청하는 흐름이 자연스러운가?
- AI가 답변하지 못하는 상황을 납득하는가?
- 사용자가 자신의 입력 정보가 어떻게 쓰이는지 이해하는가?

AI 서비스의 피드백은 기능 만족도뿐 아니라 신뢰와 책임 범위를 봐야 합니다.

피드백 반영 전후 비교

피드백은 반영 전후를 비교해야 의미가 있습니다.

구분	반영 전	반영 후
첫 화면	사용자가 시작 버튼을 찾지 못함	버튼 문구를 “오늘 추천받기”로 변경
입력 화면	알레르기 입력 이유를 모름	“안전한 추천을 위해 필요해요” 설명 추가
결과 화면	추천을 믿기 어렵다고 느낌	추천 이유와 제외 기준 표시
오류 상황	결과 없음 후 막힘	다음 수정 버튼을 일찍 제안 추가

노류 양형	결과 없음 부 검증	소인 수정 머는과 관와 세인 수기
-------	------------	--------------------

개선은 “바뀌다”가 아니라 “사용자 반응이 나아졌는가”로 판단합니다.

프로토타입 검증 결과를 정리하는 방법

검증 후에는 다음을 정리해야 합니다.

항목	질문
확인된 점	어떤 가설이 맞았는가?
발견된 문제	사용자가 어디서 막혔는가?
원인 해석	왜 그런 문제가 생겼는가?
개선 결정	무엇을 바꿀 것인가?
보류 결정	지금 반영하지 않을 것은 무엇인가?
다음 검증	다음에는 무엇을 확인할 것인가?

검증 결과는 다음 버전의 기획 근거가 됩니다.

반영하지 않을 피드백도 정해야 한다

사용자 피드백은 모두 소중하지만 모두 반영할 수는 없습니다.

반영하지 않을 수 있는 경우:

- 핵심 사용자와 거리가 먼 요구
- MVP 범위를 크게 벗어나는 기능
- 한 사람에게만 나타난 취향 문제
- 기술이나 운영 부담이 너무 큰 변경
- 더 많은 검증이 필요한 의견

중요한 것은 무시가 아니라 판단입니다.

왜 지금 반영하지 않는지 설명할 수 있어야 합니다.

프로토타입 검증에서 흔한 실수

다음 실수를 조심해야 합니다.

- 화면을 설명해 주면서 검증한다
- 사용자가 좋다고 한 말만 기록한다
- 한 명의 강한 의견에 전체 방향을 바꾼다
- 모든 피드백을 기능 추가로 해석한다
- 오류와 빈 결과 화면을 검증하지 않는다
- AI 결과의 신뢰 문제를 디자인 문제로만 본다
- 피드백 반영 후 다시 확인하지 않는다

검증의 목적은 칭찬을 받는 것이 아니라 더 나은 판단을 얻는 것입니다.

AI 기획자의 프로토타입 점검 질문

프로토타입을 볼 때 다음 질문을 던져 봅니다.

- 사용자가 첫 화면에서 서비스 목적을 이해하는가?
- 핵심 흐름이 너무 길거나 복잡하지 않은가?
- AI가 필요한 순간이 분명한가?
- AI 결과의 이유와 한계가 보이는가?
- 사용자가 결과를 수정하거나 다시 요청할 수 있는가?
- 빈 결과, 오류, 안전 제한 상태가 준비되어 있는가?
- 피드백을 반영한 뒤 무엇이 개선되었는가?

프로토타입 개선의 반복 구조

프로토타입은 한 번 만들고 끝나는 것이 아닙니다.

가설 설정
→ 프로토타입 제작
→ 사용자 반응 관찰
→ 피드백 정리
→ 개선 방향 결정
→ 프로토타입 수정
→ 다시 검증

이 반복을 통해 서비스는 점점 사용자에게 맞는 형태로 다듬어집니다.

오늘의 정리

- 프로토타입은 완성품이 아니라 서비스 가설을 검증하기 위한 도구다
- 프로토타입의 목적은 화면 완성도보다 사용자 이해, 흐름, 가치, 신뢰를 확인하는 데 있다
- AI 서비스 프로토타입은 결과 설명, 수정, 재시도, 오류 상태까지 함께 확인해야 한다
- 사용자 피드백은 말과 행동을 함께 보고 문제 유형별로 정리해야 한다
- 피드백 반영은 모든 요청을 넣는 것이 아니라 핵심 문제를 찾아 우선순위에 따라 개선하는 과정이다

마지막으로 기억할 문장

프로토타입은 정답을 보여주는 것이 아닙니다.

사용자 앞에서 우리의 가정을 확인하고,
틀린 부분을 빨리 배우고,
다음 버전을 더 낮게 만드는 도구입니다.

강사님이 반복해서 강조하는 핵심 포인트 모음

각 자료 말미의 “오늘 기억할 문장” / “마지막으로 기억할 문장”과, 여러 자료에 걸쳐 반복 등장하는 원칙들을 모았습니다.

자료별 “기억할 문장”

자료	기억할 문장
시장조사	좋은 시장조사는 자료 수집이 아니라, 사용자 문제와 서비스 기회를 찾는 과정입니다.
문제 정의(Pain Point)	Pain Point는 기능 목록이 아니라, 사용자가 실제로 겪는 불편함입니다.
가설수립 및 KPI 설계	가설은 검증할 대상이고, KPI는 그 검증 기준이며, AI 적용 가능성 검토는 그 해결책이 AI여야 하는 이유입니다.
AI 서비스와 데이터의 이해	AI 기획자는 데이터를 직접 만드는 사람보다, AI가 믿고 쓸 수 있는 데이터인지 확인하는 사람에 가깝습니다.
데이터 품질과 전처리	좋은 AI는 좋은 모델만으로 만들어지지 않습니다. AI가 믿고 사용할 수 있는 데이터가 있어야 좋은 서비스가 됩니다.
데이터 정의서	데이터 정의서는 개발자를 위한 표가 아니라, 기획자·개발자·디자이너·운영자가 같은 데이터를 같은 의미로 이해하게 만드는 약속입니다.
API 설계와 AI 서비스 연동	API 정의서는 개발자만을 위한 문서가 아닙니다. AI 기획자가 서비스 기능과 데이터, AI 답변 흐름을 연결하기 위해 필요한 약속입니다.
MVP와 서비스 기능 정의	첫 버전의 목표는 완벽한 서비스를 만드는 것이 아니라, 가장 중요한 사용자가 가장 중요한 문제를 가장 짧은 흐름으로 해결할 수 있는지 확인하는 것입니다.
정보구조(IA) 및 User Flow	서비스 구조는 회사가 만든 기능의 목록이 아닙니다. 사용자가 자기 문제를 해결하기 위해 이해하고, 선택하고, 이동하는 길입니다.
서비스 시나리오 및 예외 처리	좋은 서비스는 사용자가 성공할 때만 친절한 것이 아닙니다. 사용자가 막혔을 때 무슨 일이 생겼는지 알려 주고, 다시 앞으로 갈 수 있는 길을 보여 줍니다.
프로토타입 구현 및 검증	프로토타입은 정답을 보여주는 것이 아닙니다. 사용자 앞에서 우리의 가정을 확인하고, 틀린 부분을 빨리 배우고, 다음 버전을 더 낮게 만드는 도구입니다.

전체 자료를 관통하는 반복 원칙

- “무엇을 만들지”보다 “왜 필요한지”가 먼저다** 문제 정의, MVP, User Story, IA 자료 모두 공통적으로 해결책(기능)을 먼저 정하지 말고 사용자 문제를 먼저 정의하라고 강조합니다. (“AI 챗봇을 만들자”는 문제 정의가 아니다 / 기능은 수단이고 사용자 가치는 목적이다)
- AI는 목적이 아니라 수단이다** 가설수립·KPI, MVP, 프로토타입 자료에서 공통적으로 “AI로 할 수 있다”와 “AI로 해야 한다”는 다르다는 점을 강조합니다. 규칙 기반으로 충분한 문제에는 AI를 쓰지 않아도 됩니다.
- 모든 주장은 정량적으로 검증 가능해야 한다** 가설·KPI 자료에서 “더 편해질 것이다” 같은 측정 불가능한 표현을 흔한 실수로 지적하며, 목표 수치와 기간을 반드시 넣으라고 강조합니다. 요구사항정의서 자료도 “빠르게”보다 “응답시간 2초 이내”처럼 정량적 기준을 명시하라고 강조합니다.
- 데이터는 출처·최신성·품질을 항상 의심하라** 데이터 요구사항 분석, AI 서비스와 데이터의 이해, 데이터 품질과 전처리, 데이터 정의서 네 자료 모두 “이 데이터는 어디서 왔는가, 언제 갱신되는가, 사용해도 되는가”를 반복해서 확인하도록 요구합니다.
- 공급자 관점이 아니라 사용자 관점으로 표현하라** MVP(User Story), IA 자료에서 “AI 추천 기능을 제공한다”가 아니라 “사용자가 선택하기 어려운 상황에서 적절한 후보를 빠르게 고른다”처럼 사용자 관점 문장으로 바꾸는 훈련을 반복 강조합니다.
- 성공 흐름만이 아니라 실패·예외 흐름까지 설계하라** IA/User Flow, 서비스 시나리오, 프로토타입 자료 모두 “성공했을 때”뿐 아니라 입력 누락, 결과 없음, AI 응답 실패, 안전 제한 상황까지 미리 설계하라고 강조합니다.
- 문서는 협업을 위한 “약속”이다** 요구사항정의서, 데이터 정의서, API 정의서 자료 모두 문서의 목적이 “정리”가 아니라 “같은 의미로 이해하기 위한 약속”이라는 점을 공통적으로 명시합니다.
- 사용자 피드백은 있는 그대로 반영하지 말고 해석하라** 프로토타입 자료에서 “검색 기능이 있으면 좋겠다”는 요청을 그대로 반영하지 말고, 그 뒤에 있는 진짜 문제(메뉴 구조 혼란, 결과 과다 등)를 찾으라고 강조합니다.

자료 곳곳의 “흔한 실수” 모음

- 검색 결과를 그대로 붙여 넣고 해석하지 않는다 (시장조사)
- 경쟁사의 장점만 나열하고 약점을 찾지 않는다 (시장조사)
- 문제보다 기능을 먼저 정한다 (문제정의)
- “불편하다”라고만 쓰고 상황을 설명하지 않는다 (문제정의)
- 해결책만 정하고 검증할 가설을 세우지 않는다 (가설/KPI)
- KPI에 목표 수치와 기간을 넣지 않는다 (가설/KPI)
- 기능만 정의하고 필요한 데이터를 적지 않는다 (데이터 요구사항)
- 데이터 출처를 “인터넷”처럼 모호하게 적는다 (데이터 요구사항)
- 빈값·이상치를 이유 확인 없이 삭제하거나 방치한다 (데이터 품질)
- 화면을 설명해 주면서 검증한다 → 사용자의 자연스러운 반응을 왜곡시킨다 (프로토타입)
- 한 명의 강한 의견에 전체 방향을 바꾼다 (프로토타입)
- 오류 상황·빈 결과 화면을 검증하지 않는다 (프로토타입)

